



PDF Download
3671147.pdf
17 December 2025
Total Citations: 15
Total Downloads:
1338

Latest updates: <https://dl.acm.org/doi/10.1145/3671147>

RESEARCH-ARTICLE

ZTA-IoT: A Novel Architecture for Zero-Trust in IoT Systems and an Ensuing Usage Control Model

SAFWA AMEER, The University of Texas at San Antonio, San Antonio, TX, United States

LOPAMUDRA PRAHARAJ, Tennessee Technological University, Cookeville, TN, United States

R. S. SANDHU, The University of Texas at San Antonio, San Antonio, TX, United States

SMRITI BHATT, Purdue University System, West Lafayette, IN, United States

MAANAK GUPTA, Tennessee Technological University, Cookeville, TN, United States

Open Access Support provided by:

Tennessee Technological University

The University of Texas at San Antonio

Purdue University System

Published: 16 August 2024

Online AM: 17 June 2024

Accepted: 03 May 2024

Revised: 22 March 2024

Received: 28 March 2023

[Citation in BibTeX format](#)

ZTA-IoT: A Novel Architecture for Zero-Trust in IoT Systems and an Ensuing Usage Control Model

SAFWA AMEER, Computer Science, The University of Texas at San Antonio, San Antonio, United States

LOPAMUDRA PRAHARAJ, Tennessee Tech University, Cookeville, United States

RAVI SANDHU, Computer Science, The University of Texas at San Antonio, San Antonio, United States

SMRITI BHATT, Purdue University System, West Lafayette, United States

MAANAK GUPTA, Tennessee Tech University, Cookeville, United States

Recently, several researchers motivated the need to integrate Zero Trust (ZT) principles when designing and implementing authentication and authorization systems for IoT. An integrated Zero Trust IoT system comprises the network infrastructure (physical and virtual) and operational policies in place for IoT as a product of a ZT architecture plan. This article proposes a novel Zero Trust architecture for IoT systems called ZTA-IoT. Additionally, based on different types of interactions between various layers and components in this architecture, we present ZTA-IoT-ACF, an access control framework that recognizes different interactions that need to be controlled in IoT systems. Within this framework, the article then refines its focus to object-level interactions, i.e., interactions where the target resource is a device (equivalently a thing) or an information file generated or stored by a device. Building on the recently proposed Zero Trust score-based authorization framework (ZT-SAF), we develop the object-level Zero Trust score-based authorization framework for IoT systems, denoted as ZTA-IoT-OL-SAF, to govern access requests in this context. With this machinery in place, we finally develop a novel usage control model for users-to-objects and devices-to-objects interactions, denoted as UCON_{IoT}. We give formal definitions, illustrative use cases, and a proof-of-concept implementation of UCON_{IoT}. This article is a first step toward establishing a rigorous formally defined score-based access control framework for Zero Trust IoT systems.

CCS Concepts: • **Security and privacy** → **Information flow control**;

Additional Key Words and Phrases: Access control, zero trust, cloud-enabled IoT, UCON

ACM Reference Format:

Safwa Ameer, Lopamudra Praharaaj, Ravi Sandhu, Smriti Bhatt, and Maanak Gupta. 2024. ZTA-IoT: A Novel Architecture for Zero-Trust in IoT Systems and an Ensuing Usage Control Model. *ACM Trans. Priv. Sec.* 27, 3, Article 22 (August 2024), 36 pages. <https://doi.org/10.1145/3671147>

This work is supported by NSF CREST-PRF Award 2112590 and NSF CREST Grant HRD1736209.

Authors' Contact Information: Safwa Ameer, Computer Science, The University of Texas at San Antonio, San Antonio, Texas, United States; e-mail: safwa.ameer@gmail.com; Lopamudra Praharaaj, Tennessee Tech University, Cookeville, Tennessee, United States; e-mail: lpaharaj42@tntech.edu; Ravi Sandhu, Computer Science, The University of Texas at San Antonio, San Antonio, Texas, United States; e-mail: ravi.sandhu@utsa.edu; Smriti Bhatt, Purdue University System, West Lafayette, Indiana, United States; e-mail: smbhatt@purdue.edu; Maanak Gupta, Tennessee Tech University, Cookeville, Tennessee, United States; e-mail: mgupta@tntech.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2471-2566/2024/08-ART22

<https://doi.org/10.1145/3671147>

1 Introduction

The **Internet of Things (IoT)** integrates the physical and digital worlds seamlessly. This transformative technology creates a smart ecosystem where everything continuously communicates and autonomously triggers actions. Various application areas, such as smart homes, health care, and manufacturing, make this technology an integral part of our daily lives. In addition to stand-alone computers, IoT objects include everyday objects that can be read, located, addressed through information sensing devices, and controlled via the Internet. These objects range from simple electronic toys to sophisticated smart vehicles. In addition, they include things that we would not usually think of as electronic, such as food, clothing, animals, trees, human bodies, and buildings. The number of connected smart devices is projected to reach over 25 billion by 2025 [20, 69]. As a new emergent technology, IoT faces challenging security and privacy concerns beyond those in traditional domains due to its complex environment, constrained devices, and heterogeneity [42, 47, 54]. Furthermore, using several other supporting technologies (e.g., cloud computing, machine learning) in this smart vision presents several security and privacy challenges [47, 54]. It is widely agreed that authentication and authorization are critical to the success of IoT [5, 12, 20, 32, 33, 37, 39, 42, 47]. In response, several researchers have recently motivated the need to incorporate **Zero Trust (ZT)** [56] principles and concepts when designing and enforcing IoT access control systems [10, 26, 30, 31, 57]. ZT focuses on authentication, authorization, and shrinking implicit trust zones to enhance security and privacy. Few solutions have been proposed in the literature converging ZT paradigms with IoT systems. These solutions take a disjointed approach, concentrating on a specific type of communication or network layer. There is still a lack of a holistic approach to address IoT security concerns, spanning multiple layers of the network and multiple security functions [30].

This article takes initial steps toward designing and enforcing comprehensive and mature ZT IoT integrated systems. It explores the security concerns and different types of interaction across multiple layers in IoT and proposes access control oriented architecture, a framework of interactions, and an authorization policy model. Section 2 provides motivation, research challenges, and background. In Section 3, we: (i) present our novel Zero Trust architecture for IoT systems (denoted as ZTA-IoT), (ii) describe ZTA-IoT-ACF, an access control framework of interactions based on ZTA-IoT, and (iii) introduce our proposed score-based authorization access control framework for object-level interactions in ZTA-IoT (denoted as ZTA-IoT-OL-SAF). With this context, Section 4 develops a novel usage control model for users-to-objects and devices-to-objects interactions, denoted as $UCON_{IoT}$, with two use cases and a proof-of-concept implementation discussed respectively in Sections 5 and 6. Section 7 provides an overview of related work. Section 8 discusses our findings and gives our conclusions.

2 Motivation, Research Challenges, and Background

2.1 Motivation

As some researchers recently suggested [10, 21, 50], to ensure the successful implementation of IoT systems, we need to rethink access control architectures, policy models, and mechanisms in light of some IoT systems' unique characteristics. Following are some of these characteristics: (i) *Heterogeneity*. Characterizes IoT device diversity in terms of size, function, communication, and networking methods. This diversity, stemming from numerous vendors employing various platforms and protocols, broadens the attack surface and introduces novel security vulnerabilities [47, 54]. (ii) *Distributed and Remote Location*. The owner of IoT devices may not have physical access to the devices as these devices can be dispersed and located elsewhere. As a result, a number of security vulnerabilities are introduced, including insecure access to the web, backend APIs, cloud, and mobile

interfaces. Each interface in this complex IoT ecosystem should be subject to access control. Commercial IoT frameworks, however, do not meet these requirements [54]. (iii) *Massive Scale*. With the proliferation of IoT ecosystems, the number of devices that must be managed while communicating with one another is increasing exponentially. Vast amounts of data are generated, and managing and securing these data has become increasingly challenging. (iv) *Autonomicity*. Using artificial intelligence along with cloud resources, IoT devices can be autonomous when deployed. (v) *Dynamic Behavior*. IoT devices' behavior changes dynamically based on user characteristics and usage context. (vi) *Connectivity and Interconnectivity*. Connectivity facilitates network accessibility and compatibility. Interconnectivity refers to the ability to connect anything to anything and to the global information and communication network. Due to this characteristic, IoT has a broader attack surface. (vii) *Things-Related Services*. IoT enables things to provide services within their constraints, such as privacy protection and maintaining semantic consistency between physical objects and their associated virtual things. For things-related services to be provided within the constraints of things, both physical and information technologies must be revised. (viii) *Safety*. IoT systems include many safety-critical services and apps, for instance, smart car apps, smart health services, and smart locks devices. It's critical to develop a scalable security framework to secure endpoints, networks, and data moving across all elements of IoT to protect our personal data and physical well-being.

The ultimate goal in IoT is to establish an autonomous smart ecosystem where everything is connected, continuously communicating, sharing information, and triggering actions. To illustrate the complexity of IoT systems, consider the use case—illustrated in Figure 1—of a smart factory, the workpieces, processing stations, and deployed applications are interconnected to exchange data, enabling factory workers to monitor and manage the manufacturing process. Data are continuously exchanged amongst entities or pushed to the cloud, and multiple factories exchange data for collaboration. Vehicles continuously update different entities with real-time delivery and supply-chain-related information. The cloud collects data from different entities, uses applications to generate insights, and provides it to various entities. These entities include apps, cloud services, smart devices, factory workers, on-site or working-from-home managers, engineers, and researchers. In this use case, we have resources from different vendors in different locations. Users, devices, and services with different privileges access those resources to perform various tasks. It is important to note that as widely recognized [4, 14, 22, 51, 53, 55], we consider cloud technologies as integral to IoT systems. IoT systems encompass pervasive heterogeneous objects with limited computational, storage, and battery capacities. Cloud technologies enable low-cost, on-demand, high-capacity storage and computing, and ubiquitous resources usable from everywhere. The inadequacies in applying access control policies in IoT applications lead devices and apps to be easily exploited to gain unauthorized access to devices and data. Moreover, the complexity of IoT systems has outstripped legacy methods of perimeter-based network security since there is no easily identifiable perimeter for an IoT system. It is imperative that a systematic and dynamic research approach is used for IoT to ensure its sustainability in securing access, authorization, communication, and data flow over the long term.

The critical question is, how can we trust such a complex and evolving infrastructure? Recently, some researchers have suggested that a possible solution is to incorporate the recently proposed **Zero Trust (ZT)** cybersecurity paradigm in IoT [10, 26, 30, 31, 57]. ZT emerged as a response to enterprise trends that include remote users, bring your own device, and IaaS/SaaS cloud services [56]. These trends are also prevalent in IoT systems, along with additional complexities. ZT assumes no implicit trust accrues to assets or user accounts solely based on their physical or network location. **Zero Trust architecture (ZTA)** is an enterprise's cybersecurity plan that utilizes ZT concepts and encompasses component relationships, workflow planning, and access policies [56]. An integrated

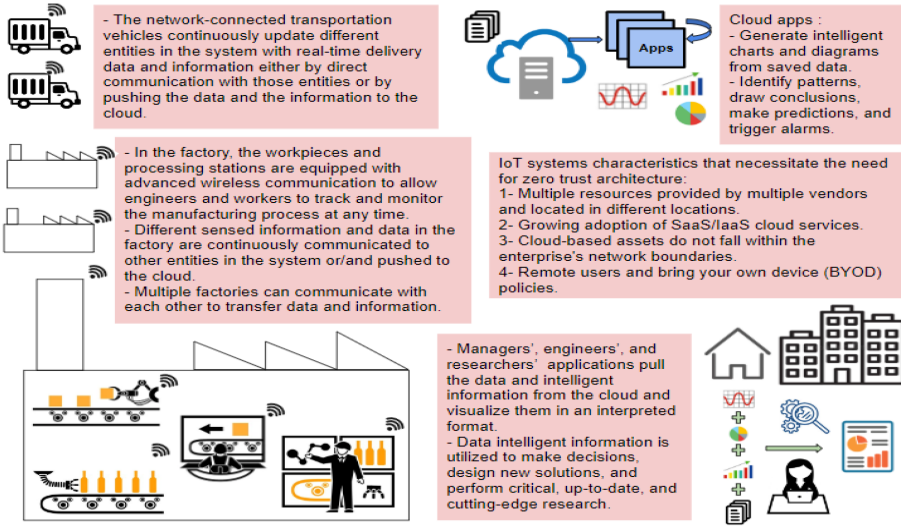


Fig. 1. IoT ecosystem use case.

Zero Trust IoT system comprises the network infrastructure (physical and virtual) and operational policies in place for an IoT system resulting from a ZTA plan [10].

2.2 Research Challenges

In [10], the authors analyzed the ZT basic tenets provided by the NIST document special publication [56] and accordingly proposed the **zero trust authorization requirements framework (ZT-ARF)**. This framework provides authorization policy models designers with a structured view of different authorization requirements to consider when designing a zero trust authorization policy model. According to the authors, these requirements can be divided into seven categories, as follows: (i) *Actor Characteristics*. The actor is the system entity that initiates the access request. It includes users, subjects, devices, and services. Actor characteristics represent the actor's properties, such as ID, clearance, roles, and the like. (ii) *Target Characteristics*. Targets are the resources that need to be accessed in the request. They include information containers, devices, services, and applications. Users and sessions can be considered as targets in certain domains like social computing. Characteristics of targets include location, owner, security posture, and the like. (iii) *Action Characteristics*. An action is an executable image of a program. For example, operations. Various characteristics can be linked to actions. For instance, within the context of smart homes, we may want to characterize kids-friendly operations. (iv) *Action-Target Characteristics*. An approval to perform an action on a specific target gives us the action target combination. For instance, the concept of permission in RBAC systems [59], and the concept of activities in activity-centric access control systems [38, 49]. (v) *Context Characteristics*. This includes environment characteristics, system characteristics, threats, and log information. (vi) *Usage Check*. This requirements component represents the continuity of an access decision, and it implies that the authorization system enforces the security policy before the access execution, during the execution, and afterward. (vii) *Behavioral Check*. This element emphasizes incorporating behavioral evaluations into the authorization process to capture any abnormal behavior.

Moreover, motivated by different IoT characteristics which are mentioned in Section 2.1, the authors in [10] introduced the following requirements for authorization models in IoT systems. Considering these requirements when designing or adapting access control models for IoT is

necessary. These requirements are: (i) *Dynamic Authorization*. Authorization models within such a highly dynamic environment should be dynamic so as to capture actor, target, and contextual information. For example, in a smart health domain, access privileges alter based on user roles, utilized devices, accessed documents, and contextual factors such as time and location. (ii) *Fine-Grained Authorization*. This is crucial in IoT due to the risk asymmetry among device operations. For example, in smart homes, “oven.on” risks fire, while “oven.off” risks uncooked food. The authorization model should be able to allow a subset of the device capabilities, avoiding unrestricted access. This requires capturing action and action-target characteristics. (iii) *Suitable for Constrained Smart Devices*. Given limited computing power and storage in smart objects and the absence of a complete IoT interoperability standard, access control models mustn’t burden these devices with extensive computation or communication requirements. (iv) *Compliance*. IoT currently lacks access control standards. It is necessary to develop new standards and models of access control that adhere to these standards to make smart communities successful and sustainable. (v) *Formal Policy Definition*. The model should have a formal definition to make the intended behavior precise and rigorous. (vi) *Scalability*. Access control mechanisms and models must be scalable to manage authorizations for both few and numerous devices, users, and entities within the IoT system. (vii) *Privacy-Preserving*. A privacy-focused access control model empowers users with control over their data and devices, allowing them to dictate their usage and uphold a user-centric privacy approach. (viii) *Continuous Ongoing Authorization*. Within IoT systems, various elements like users, devices, services, and apps communicate in specific contexts (e.g., lawnmower operation, TV watching) with interconnected, synchronous, and temporally continuous interactions. This mandates policies with contextual awareness and full access life cycle management. IoT access control models should exhibit dynamic qualities for continuously verifying authorization policies, ensuring sustained control for relatively long-lived operations or immediate revocation. (ix) *Scored-Based Authorization*. Authorization models can be characterized based on how input factors evaluate access requests. Accordingly, two models are recognized: criteria-based and score-based. Criteria-based requires meeting specific qualifications before granting the resource access (e.g., RBAC, UCON, ACON, ABAC). Alternatively, score-based models calculate a confidence level (score) for each request. Access is granted when the score surpasses a resource’s threshold; lower scores may decline or restrict access. Dynamic IoT systems with human-device-data interactions within a dynamic environment demand adaptive access rights. Moreover, many IoT devices lack security certifications, risking unknown threats. Additionally, sensor-derived inputs often hold subjectivity and probability. Thus, authorization policies must incorporate confidence levels (scores) for access requests to accommodate subjective data and uncertainty.

Based on the above IoT authorization requirements and the ZT-ARF, Ameer et al. [10] have recently demonstrated that to address the security challenges in IoT systems, the need arises for integrated ZT IoT system plans, frameworks, and architectures that enable the development of contextual aware access control models capable of: (i) incorporating actor, targets, action, action-target, and context characteristics, (ii) continuously performing ongoing authorization, and (iii) dynamically deciding on access requests based on calculated score (confidence level) rather than on static criteria-based access control policies.

2.3 Threat Model

Our threat model is based on the well-known and long-standing distinction between access-control policy and mechanism [44]. A more recent elaboration of this distinction called the PEI framework [61, 64] recognizes three distinct layers of policy models (P), enforcement models (E), and implementation (I). For our purpose in this article, it suffices to distinguish the policy and implementation layers, where implementation is essentially synonymous with the mechanism in the

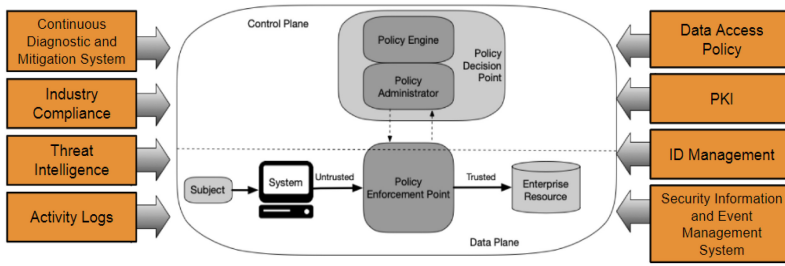


Fig. 2. Zero trust logical components [56].

vintage terminology. At the policy layer, the security architect’s primary concern is to enable specification of *what* authorizations are to be enforced and how they are managed. At the implementation layer, code and protocols have to be designed, built, and deployed to *enforce* the P layer policy requirements. In general, there are numerous practical implementations for any given P layer model. A discussion of this distinction in the context of the familiar role-based access control model [35, 66] is given in [60].

The extensive literature on threat modeling primarily addresses vulnerabilities at the implementation layer [74]. There is a lack of literature concerning threat modeling at the policy layer. There is scant literature regarding “threat modeling” at the policy layer. Various valuable policy models exist, including lattice-based information flow control [28, 65], role-based access control [35, 66], attribute-based access control [41, 67], and usage control [48]. Vulnerabilities differ greatly between the P and I layers. For example, lattice-based models, which are derived mathematically from Denning’s axioms [28], do not capture all aspects of information flow. For example, the axioms do not address inference of sensitive information by “connecting the dots” across multiple items of non-sensitive information. In contrast, role-based access control models are based on motivating principles rather than mathematical axioms [58, 62], such as privilege abstraction, separation of administration, separation of duty, and least privilege.

P models aim to facilitate authorization administration yet may not prevent deliberate or accidental misconfigurations by administrators. In P models, the main threat is twofold. First, the mathematical axioms or motivating principles may be inadequate for the model’s intended application domain. Second, there’s a risk of inadvertent or malicious authorization misconfiguration by administrators. Various models, including the one discussed in this article, integrate administrator constraints to address this latter possibility.

This article introduces a usage control model at the P layer, assuming a flawless implementation, though recognizing this ideal isn’t realistic. Numerous implementation vulnerabilities are expected. Investigating these vulnerabilities requires a concrete implementation with its inherent security weaknesses. These weaknesses in the I layer are specific to particular implementations and are not intrinsic to the P model.

2.4 Background

The ZT NIST document [56] proposed a conceptual framework for ZTA in terms of logical components and their interactions, as shown in Figure 2. These components are classified as core components and data sources. The policy decision point and policy enforcement point are core components, while the orange components are data sources. Data sources provide input and policy rules used by the policy engine to make access decisions. The ZTA logical components use a separate control plane to communicate, while application data is communicated on a data plane. These logical components should make up a ZTA deployment in an enterprise. However, as illustrated

in the IoT use case presented in Figure 1 and described in Section 2.1, the system entities requesting access or initiating an action may include users, devices, machines, vehicles, cloud services, and applications. Moreover, the resources on which actions are being performed may include information containers, applications, cloud services, virtual objects, devices, machines, and in some cases, users or users' sessions. Accordingly, the ZTA shown in Figure 2 needs to be extended and adapted to fully express and govern different types of interactions in cloud-enabled IoT systems. In Section 3.1, we present our novel ZT architecture for IoT systems called *ZTA-IoT*. This architecture extends Figure 2 to encompass different IoT entities and use cases. ZTA-IoT bridges the existing gap between NIST's ZTA framework and IoT systems.

3 ZTA-IoT and Associated Frameworks

This section presents our novel ZTA-IoT architecture and its associated ZTA-IoT-ACF and ZTA-IoT-OL-SAF access control frameworks.

3.1 ZTA-IoT: Zero Trust Architecture for IoT

In the last decade, several researchers have proposed layered IoT architectures, encompassing Object, one or more Middle, and Application Layers [4, 6, 14, 18, 51, 72]. These layers contain components like users, IoT devices, and applications, which may be compromised. These components, hailing from diverse vendors and locations, underscore the need for integrating ZT principles into IoT architectures. However, the current gap between ZT and IoT architectures is evident. While NIST's developed ZTA [56] lacks IoT-specific components and interactions, a comprehensive ZTA for IoT must include not only IoT devices but also cloud and application components.

In this section, we develop a novel **ZTA for IoT systems (ZTA-IoT)** primarily focusing on cloud-enabled IoT.

Our proposed ZTA-IoT is illustrated in Figure 3, which incorporates the concepts of Figure 2 while adding significant extensions to accommodate IoT and Cloud components. The fundamental distinction between the data plane and control plane is carried over from Figure 2, but is now shown as a left-right distinction. The entirety of the control plane of Figure 2 now comprises the entirety of the right-hand side of Figure 3. The left-hand side of Figure 3 recognizes the layered architecture of cloud-enabled IoT. The ZTA-IoT aligns with prior layered IoT architectures in this regard with four layers: *Object Layer*, *Virtual-Object Layer*, *Cloud Layer*, and *Application Layer*. Different interactions can occur between different components within these layers to exchange data and commands so as to accomplish specific tasks. For instance, in a smart home scenario, multiple sensors and devices can communicate autonomously to perform a task (e.g., turn on lights when it is dark, adjust thermostat temperature based on temperature sensors, and open all the ventilation devices when there is a fire alarm). To ensure overall security and privacy within ZTA-IoT for any IoT domain, it is essential to secure all these communications. Therefore, we follow the ZT principles in this architecture where none of the components or resources are inherently trusted; thus, components in one layer cannot communicate directly with components in the same/other layer/layers. Additionally, components in different layers cannot be accessed directly by users. Each communication has to go through the policy enforcement layer and the policy decision layer. The policy enforcement layer ensures that only authorized accesses by the policy decision layer are allowed.

We discuss different ZTA-IoT layers below, starting with the Policy Enforcement and Policy Decision layers.

Policy Enforcement Layer. This layer enables and manages all interactions between IoT entities that are spread across other layers. It overlaps the data and control planes. It incorporates different **policy enforcement points (PEPs)**, which communicate with the policy administrator to

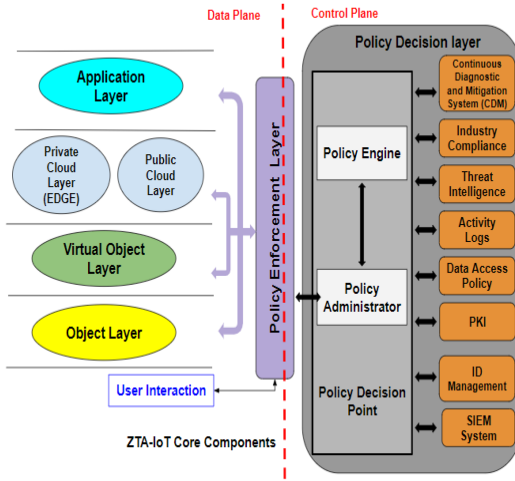


Fig. 3. ZT architecture for IoT systems (ZTA-IoT).

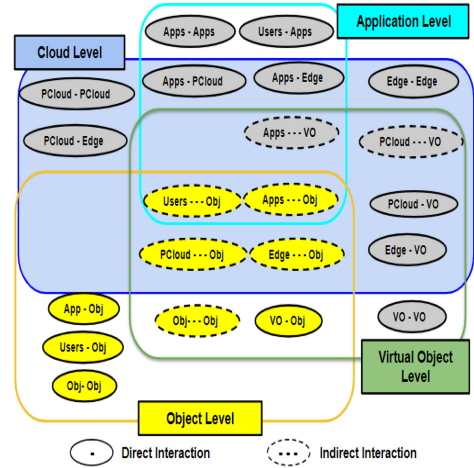


Fig. 4. ZTA-IoT-ACF, access control framework based on different interactions between the ZTA-IoT layers.

forward requests for decisions and receive policy updates. Each PEP is a logical component consisting of two different parts: the actor side (e.g., an agent on a laptop) and the target side (e.g., a gateway that controls access in front of the target). Alternately a PEP can be a single component that acts as a mediator for communication from a sender to a receiver.

Policy Decision Layer. This layer comprises the **policy decision point (PDP)**, which includes the **policy engine (PE)**, **policy administrator (PA)**, and potentially additional inputs for the PE. The PE, responsible for permitting or denying access to a resource target, may utilize multiple data sources to inform its decisions. The PA establishes and terminates communication paths between an actor and a resource target by instructing relevant PEPs. In Figure 3, the orange elements represent the data sources within the policy decision layer. These data sources include: (i) *Continuous Diagnostics and Mitigation (CDM) System*. Provides real-time updates on the state of IoT entities, including vulnerabilities and malware. Moreover, this system recognizes and enforces policies on enterprise and non-enterprise devices using enterprise infrastructure. (ii) *Industry Compliance System*. Ensures adherence to industry standards and encompasses enterprise compliance policies. (iii) *Threat Intelligence*. This offers insights into IoT entity-related vulnerabilities and attacks. (iv) *Activity Logs*. This provides information regarding the security posture of the IoT network and entities in the system. (v) *Data Access Policy*. Encompasses authorization rules and attributes used in making access control decisions. (vi) *Public Key Infrastructure (PKI)*. Manages certificates and keys for IoT entities. (vii) *ID Management System*. Creates and manages identity records for IoT components, including user or thing names, roles or attributes, and the like. (viii) *Security Information and Event Management (SIEM) System*. Collects security data about IoT entities to be used in refining access control policies and identifying malicious activities.

The layers that comprise the data plane of a cloud-enabled IoT system are as follows:

Object Layer. This layer includes heterogeneous types of IoT objects, such as sensors, actuators, devices, and the like. These objects can operate individually or as clusters, gathering and transmitting data to upper architecture layers. Clustered objects often possess built-in applications (e.g., tire pressure monitoring in smart vehicles and smart house's cameras' monitoring system). These

applications can communicate with different clustered objects (or sometimes outside the cluster objects) to retrieve or update data. Some applications accumulate and send data to the cloud for further analysis.

Virtual Object Layer. This layer incorporates the digital counterparts of physical IoT objects. These digital representations provide a current persistent status of objects when physical and virtual objects are connected, or otherwise the desired future status or the last received status if they are not connected. **Virtual Objects (VOs)** address IoT challenges like scalability, heterogeneity, security, privacy, and identification. VOs enable seamless communication regardless of object differences and location. VOs can be associated with physical objects in various ways (one to one, many to one, or one to many). This layer also facilitates interactions between IoT objects to applications via VOs. It offers an interface where users can easily communicate with objects and visualize the analyzed information. General users and administrators can communicate remotely with virtual objects through applications. VOs can be created at the edge or in a central cloud, as appropriate.

Cloud Layer. This layer includes both large aggregated public cloud platforms (**Public Cloud Services Layer**) and local private edge cloud (**Private Cloud Layer (Edge)**). This layer enables data storage, computation, and processing for large data collected from IoT objects, whether centrally in public clouds or locally in edge clouds. The data stored at this layer can be used intelligently for smart monitoring and actuation, and it can also be represented and visualized in meaningful ways for the users. Multiple cloud platforms can also communicate to share information at a broad level and pursue common goals. The cloud services running at this layer can access information and data from virtual objects.

Application Layer. This layer is the topmost layer of the IoT architecture. It delivers services and system functionalities to the end users. It offers an interface where users can easily interact with objects and visualize the analyzed information meaningfully. It can be seen that physical objects communicate with virtual objects, which in turn push these data to the cloud where applications can access it.

3.2 ZTA-IoT-ACF for Access Control

As illustrated in Section 3.1, different components within different ZTA-IoT layers need to communicate to accomplish various tasks. Keeping all these communications and interactions secure is crucial for ensuring the overall security and privacy of ZTA-IoT. Therefore, as depicted in Figure 4, we propose an access control framework for ZTA-IoT (ZTA-IoT-ACF). This framework captures various pairwise interactions that occur between different components within and across different ZTA-IoT layers. These interactions are categorized into *Object Level*, *Virtual-Object Level*, *Cloud Level*, and *Application Level* interactions. The object level category covers all interactions when the requested resource is an object, and these are highlighted in yellow in the figure. Similarly, the virtual object level, cloud level, and application level categories cover all the interactions where the requested resource is a virtual object, a cloud service, or an application, respectively. To avoid clutter in the figure, we assume that the supported interactions are symmetric so, for example, *VO-Obj* in Figure 4 implies *Obj-VO* is also included. The ZTA-IoT-ACF provides an initial foundation for creating secure access control models suited for the diverse and evolving interactions in integrated ZT IoT systems. Researchers can utilize it to explore access control needs and attributes within ZTA-IoT, guiding the development of suitable policy models, enforcement structures, and implementation strategies. This article focuses on object level interactions while recognizing that; eventually, we must investigate and integrate virtual-object, cloud, and application level interactions. We choose object-level as our first step due to its centrality to IoT.

As mentioned earlier, physical objects communicate with virtual objects, which in turn, push data to the cloud so that applications or users through applications can access these data. Accordingly, we have two types of interactions, direct and indirect, shown with solid and dashed lines, respectively, in Figure 4. A direct interaction is necessarily within the same layer, e.g., *Obj-Obj*, or across adjacent layers, e.g., *VO-Obj* or *Obj-VO*. Any interaction or communication beyond adjacent layers is necessarily indirect since it is facilitated through other layers, e.g., *Apps--Obj*. Entities within the same or adjacent layers can interact directly but may also do so indirectly. For example, indirect interaction from obj_1 to obj_2 denoted obj_1--obj_2 could happen via the sequence of direct interactions obj_1-vo_1 , vo_1-vo_2 and vo_2-obj_2 .

Object Level Interactions. Figure 4 recognizes four direct object-level interactions shown as solid yellow ovals, viz. *Obj-Obj*, *VO-Obj*, *Users-Obj*, and *Apps-Obj*. *Obj-Obj* refers to direct communication between different types of IoT devices, such as sensors and actuators. For instance, sensors gather and share data with actuators to take specific actions, similarly an actuator can send messages or recommendations to a sensor with a display screen for any user (e.g., a biomedical sensor with a display screen for a patient). Thus, these communications can be one-way or bi-directional. *Obj-VO* represents interactions between physical objects and their virtual counterparts. For example, interactions between a smartwatch and its virtual counterpart where all the data collected from the smartwatch is sent to its virtual counterpart residing at the edge or cloud. *Users-Obj* refers to user interactions with physical devices, as seen in wearable smart devices. Finally, *Apps-Obj* refers to direct communication between objects' or clustered objects' and their built-in apps, such as tire pressure monitoring apps in smart vehicles interacting with tire sensors. Moreover, objects can also be indirectly accessed from other components, even across non-adjacent layers, shown as yellow dashed ovals. *Edge--Obj* represents communication between edge clouds and objects, usually involving three layers: edge to virtual object, and then virtual object to object. Thereby, this communication is positioned in the intersection area between the cloud level, the VO level, and the object level categories. Similarly, *PCloud--Obj* is an indirect communication involving three layers: the public cloud, the VO, and the object. Accordingly, it is represented as a dashed yellow oval in the intersection area between the cloud, the VO, and the object levels. In some cases, the *PCloud--Obj* may involve the public cloud, the Edge, the VO, and the object layers. The *Apps--Obj* indirect interaction indicates the indirect communication between apps in the application layer and objects in the object layer. The *Users--Obj* indirect interaction indicates the indirect communication between users (via applications) and objects in the object layer. The *Apps--Obj* and the *Users--Obj* communications encounter four layers, the application, the cloud, the VO, and the object layers. Hence, they are represented as indirect (dashed) yellow ovals in the intersection of the application, the cloud, the VO, and the object layers. Finally, the *Obj--Obj* indirect interaction indicates the indirect communication between two objects, typically via their respective virtual objects.

Virtual Object Level Interactions. This level captures all interactions associated with virtual objects, such as direct interactions, which include *VO-VO*, *Edge-VO*, *PCloud-VO*, and *VO-Obj*, and indirect interactions that include *PCloud--VO*, and *Apps--VO*. If there is a virtual object created for a physical device, then any interaction to and from that physical device passes through its associated VO. For example, a cloud service analyzes data collected from an IoT device and need to send meaningful results back to that device, so it will first send it to the device's VO and then VO will sync the data and state with the physical device.

Cloud Level Interactions. In the ZTA-IoT, the cloud layer includes large aggregated public cloud platforms and a local private edge cloud that enables local communication, computation, storage, and analysis. This level consists of the following interactions *PCloud-PCloud*, *Apps-Edge*,

Apps-PCloud, *PCloud-Edge*, and *Edge-Edge*. It also includes *VO-PCloud*, *VO-Edge*, *Obj--PCloud*, and *Obj--Edge* which are implied by the interactions *PCloud-VO*, *Edge-VO*, *PCloud--Obj*, and *Edge--Obj*, respectively. An example of a *Obj--PCloud* communication would be a smart thermostat communicating with the cloud services to analyze the smart home's temperature pattern.

Application Level Interactions. This category includes all application-related interactions in the ZTA-IoT. It includes *Apps-Apps* and *Users-Apps* communications. It also includes edge to application and public cloud to application, which are implied by the interactions *Apps-Edge* and *Apps-PCloud*, respectively. In a real-world scenario, apps are a critical aspect of the IoT ecosystem which allows users to interact with cloud or edge services, virtual objects, and IoT devices. It enables users to extract useful insights from the data collected from IoT devices that might be stored in the cloud or allows users to send commands to devices through VOs to perform specific actions. In a remote patient monitoring scenario, a doctor is using an app to monitor a patient using various smart wearable devices and sensors and analyzing data collected from these devices.

3.3 ZTA-IoT-OL-SAF: Object-Level Score-Based Authorization Framework

This section refines our focus to object-level interactions within the ZTA-IoT-ACF. It introduces a score-based authorization framework inspired by the work of Ameer et al. [10], who proposed an initial abstract framework for score-based authorization in various ZT systems (ZT-SAF). In ZT-SAF [10], access request components include: (a) Actors, representing system entities initiating action or requesting access; (b) Targets, denoting resources where actions are performed, spanning from information containers to applications, services, and devices. Additionally, it may encompass users and sessions (e.g., in the social computing domain); (c) Actions, which are executable programs or activities performed on target resources as per manufacturer specifications. The authors stressed the necessity of adapting this framework across different application domains. This section presents an expanded conceptual ZT score-based authorization framework tailored to object-level interactions within the ZT architecture for IoT systems (ZTA-IoT). In this adaptation to cover object-level interactions within the ZTA-IoT, actors can include users, devices, services, and administrators. Additionally, targets may encompass devices or information stored on or generated by devices, while actions entail operations conducted on various devices and information. The proposed ZTA-IoT-OL-SAF framework is depicted in Figure 5. In this framework, access decisions are based on a predefined access policy and accepted trust level expressed as a score. Score metrics are computed for different entities involved in the access request, and threshold values are computed for different resources. The **access decision enforcement engine (ADE)** considers the calculated scores, resource threshold, and predefined policies when granting or denying access. ZTA-IoT-OL-SAF's basic components are discussed below.

Users and Users' Sessions. Actors can be users. Users are humans directly interacting with the system. Users create sessions during which they initiate some actions in the system. One user can create multiple sessions during which he/she can perform different actions on different objects.

Devices and Devices' Sessions. In IoT systems, devices can be actors too. Each device creates one or more sessions during which it initiates some actions in the system.

System's Processes Module (SPM). This conceptual component includes all the applications', clouds', or virtual objects' processes that can be triggered to initiate actions or request access to different resources.

Administrators and Administrators' Sessions. Systems administrators can initiate multiple sessions during which they can perform actions to control the entire system. In highly distributed computing systems, such as cloud-enabled IoT, controlling administrators' access is critical.

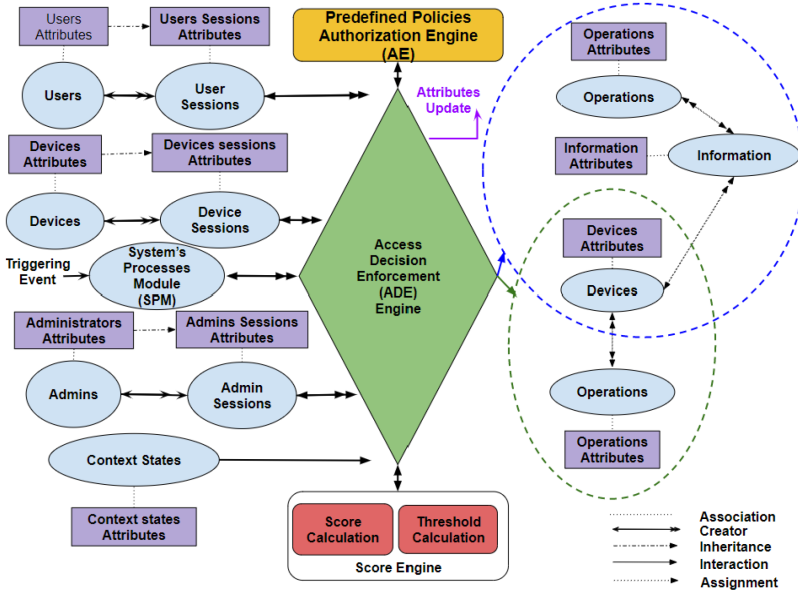


Fig. 5. Object level score-based authorization framework for ZT IoT systems (ZTA-IoT-OL-SAF).

Context States. This is a set of states that describe the system context. Each state describes the system's and the environment's context at a specific time. It may contain states like *current* and *last hour*, representing the picture of the system's and the environment's context currently and during the last hour, respectively.

Devices, Information, and Operations. In this framework, targets can be devices or information stored or generated by devices. Actors request to access targets to perform operations. We have two types of operations, operations on devices (for instance, turn on the front door camera) and operations on information (for instance, delete the video feeds files stored by the front door camera).

Predefined Policies Authorization Engine (AE). Utilizing this engine, access requests are assessed against predefined authorization policies. These policies require a formal definition language and should account for the attributes of the access request components when making access decisions. The engine can adopt various policy models per domain needs.

Score Engine. Two main functions make up this engine: score calculation and threshold calculation. This engine receives the access request from the **access decision enforcement (ADE)** engine. It uses the score and threshold calculation functions to calculate the access request score values and the resource threshold values, respectively. The access request score value is a real-time, measured determination of trust granted to the requested access. The score calculation function implements a score calculation algorithm to calculate the score of the access request. Different aspects may affect the score calculation result. These may include the attribute values of the context, the involved actor, session, target, and operation. In some cases, these aspects may include the system's historical information. On the other hand, the calculated resource threshold is a real-time, measured determination of the acceptable trust level for the requested resource at the current instant of time. The threshold calculation function implements a threshold calculation algorithm to calculate the current threshold of the requested resource. Different aspects may affect the calculated threshold, including the requested target attributes, the requested operation attributes,

and the context attributes' values of the system. Also, in some cases, these aspects may include the system's historical information. There can be different types of score and threshold calculation algorithms ranging from simple mathematical formulas to heuristic algorithms and advanced machine learning models, as recently motivated by [11]. Choosing the appropriate score engine algorithms requires a thorough investigation and a comprehensive theoretical and performance comparison between different approaches and is left as a future research direction.

Access Decision Enforcement Engine (ADE). This engine receives the actor's request and utilizes the authorization engine policy model, the score engine's calculated access request score, and the score engine's calculated resource threshold to grant or deny the requested access.

4 UCON_{IoT}: Usage Control for Users and Devices to Devices and Information

In this section, we further refine our focus to object-level interactions that accommodate users and devices as actors and devices and device information as targets. With respect to Figure 5, this narrows the actors to the top two on the left of the diagram, i.e., to users and devices. The target side of the right of the figure is unchanged. We use the term *usage control* for our model because it adopts the sophisticated perspective on authorization presented in [48], specifically including continuity of enforcement and mutability of attributes.

We describe UCON_{IoT} in two parts. In Section 4.1, we define ADE_{UCON_{IoT}}, an algorithm for the access decision enforcement engine (ADE) in Figure 5. Moreover, in Section 4.2, we propose our authorization engine predefined usage control policy model UCON_{IoT}. UCON_{IoT} is developed using the well known conceptual framework and formalism of [48]. We note that ADE_{UCON_{IoT}} and UCON_{IoT} are particular solutions for the ADE algorithm and the AE predefined policy model. Alternate solutions may be proposed for the same purpose or for different use cases. Developing the score engine algorithms and extending the actors in Figure 5 to include the bottom two on the left side (i.e., the system process module and the administrators) is beyond the scope of this article.

4.1 ADE_{UCON_{IoT}}: An ADE Algorithm

As described in Section 3.3, the ADE engine receives the access request and decides whether to grant or deny access according to its implemented algorithm and based on the AE predefined policy model and the score engine calculated score and threshold. Below, we describe the access decision engine algorithm ADE_{UCON_{IoT}} depicted in Figure 6.

The ADE_{UCON_{IoT}} algorithm performs the following steps: (i) Receives the access request; (ii) Sends the request to the score engine, which calculates the access request score and the requested resource threshold; (iii) Receives the score and the threshold values from the score engine. If the computed score is less than the calculated threshold, the access will automatically be denied, and the algorithm will finish its execution. Otherwise, the algorithm will continue to the following step. (iv) Based on the actor type (a user or a device) and the target type (a device or information), the ADE will execute the suitable check access predicate. Section 4.2 explains the check access predicate functions. If the check access predicate is not satisfied, this indicates that according to the configured UCON_{IoT} policies in the AE, the actor is not allowed to access the requested operation on the requested resource. Accordingly, the access is denied, and the algorithm will finish its execution. On the other hand, if the check access predicate is satisfied, the access is authorized, and the algorithm continues to the next step; (v) After the access is authorized, and before enforcing the requested operation, the ADE first runs the suitable pre-enforcement attributes update function (PreUpdate function). PreUpdate functions are explained and formalized in Section 4.2. Here, we have two cases: (a) An applicable PreUpdate function indicates a need to update some attributes before enforcing the access. Hence, the ADE will first enforce

the update of the attributes. Then the ADE will enforce the operation execution; (b) There is no applicable PreUpdate function; this indicates no need to update any attributes before enforcing the access. In this case, the ADE engine will enforce the execution. (vi) After enforcing the execution, we have two cases; if the requested operation is a one-time operation, for instance, increasing the thermostat temperature, then the algorithm will go to step (vii); Suppose the requested operation is a long-lived operation (a long-lived operation is continuously executed during a period of time). In that case, the algorithm needs to constantly verify the access request to facilitate ongoing controls and immediate revocation. For instance, in a smart home use case, if the time is currently 5:50 pm and kids are watching TV, they are only allowed to watch it till 6:00 pm. The system should enable continuous authorization to revoke the TV access at 6:00 pm. Hence, after enforcing the execution, the ADE will enter a loop where it keeps running the check access predicate and the suitable ongoing (during execution) attributes update function (OnUpdate function, which is explained in Section 4.2). The loop will break in three cases: (a) *Stop event*. If the execution is completed (shown in blue in Figure 6), then the ADE will go to step (vii); (b) *The ongoing attributes update function is applicable*. In other words, an ongoing (during execution) attributes update is needed. In this case, the ADE will enforce the attribute update and continue waiting for a stop event or another applicable ongoing attribute update; (c) *Another stop event is if the check access predicate denies the access (shown in red in Figure 6)*. This indicates that the access is not authorized anymore; hence, the ADE will terminate the execution and continue to the next step; (vii) The ADE will run the suitable post-execution attributes update function (PostUpdate function, which is explained in Section 4.2). If applicable, the ADE will enforce the update of the attributes, and finally, the algorithm will end execution. Otherwise, there is no need to perform post-execution attributes update; hence, the ADE engine will end its execution.

4.2 The UCON_{IoT} Policy Model for the AE

As discussed earlier and as recently proposed in [10], to meet the ZT IoT systems access control requirements, we need an authorization model that decides on every access request based on two factors: (a) A contextual aware predefined formal authorization policy model capable of capturing different attributes in the system and performing continuous authorization to facilitate ongoing controls and immediate revocation; (b) Real-time dynamically calculated score and threshold values. This section investigates the first factor, leaving the second factor for future work.

We introduce the UCON_{IoT} Policy Model, also simply called UCON_{IoT}. UCON_{IoT} is a formal usage control authorization policy model for the **authorization engine (AE)** in the ZTA-IoT-OLSAF shown in Figure 5. UCON_{IoT} governs user to device, device to device, user to information, and device to information access control. Figure 7 conceptually depicts the model components. The model's basic components' formal definition is given in Tables 3–7 in Appendix A, while described in the following:

Basic Sets and Functions.. Actors can be users (U) or devices (D). Users or devices create users' sessions (S_u) or devices' sessions (S_d), respectively, to perform some actions in the system. The session can only be terminated by the user or the device that created it. The function $user(s_u)$ is used to associate each session s_u with its unique user creator, while the function $device(s_d)$ is used to associate each session s_d with its unique device creator. Objects (OBJ) is the set of resources in the system. These resources can be devices (D) or information ($INFO$). Information is the set of information generated by or stored on different devices. They can be accessed by different actors to perform different operations on them. The function $info(d)$ returns the set of valid information generated or stored by the device d . Operation (OP) refers to the set of actions in the system. These operations can be devices' operations (OP_d) or information operations (OP_{info}) following

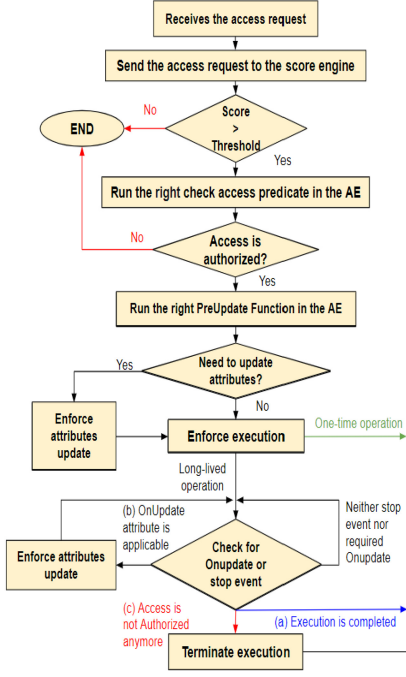


Fig. 6. $ADE_{UCON_{IoT}}$ access decision engine algorithm.

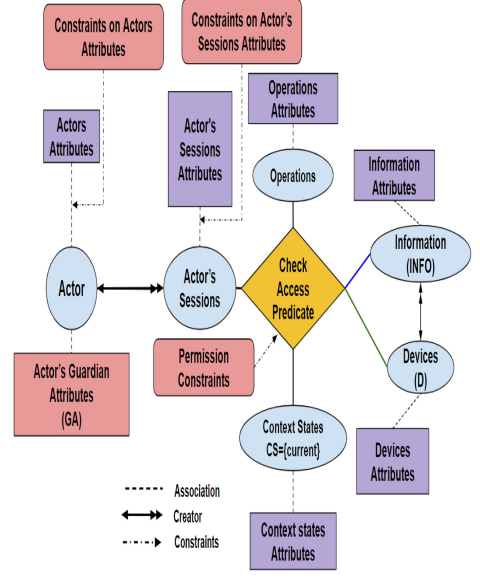


Fig. 7. $UCon_{IoT}$ policy model.

manufacturer specifications. Every device d is mapped to the set of valid operations on d by the function $ops(d)$, while every information $info$ is mapped to the set of valid operations on $info$ by the function $ops(info)$. The approval of an operation to be performed on a device or an information object is called permission. P is the set of permissions in the system. $P = P_d \cup P_{info}$. P_d is a subset of $OP_d \times D$, while P_{info} is a subset of $OP_{info} \times INFO \times D$. Context states set ($CS = \{current\}$) is a singleton set where *current* denotes the snapshot of the context at the current time. Future extensions of this component, such as $CS = \{current, yesterday, lastweek\}$, could include other time instants, such as yesterday and last week. The basic sets and functions are formally defined in Table 3 in Appendix A.

Attribute Functions. Attributes describe the characteristics of users, devices, users' sessions, devices' sessions, operations, information, and context states. An attribute is a function that takes an entity as input, such as a device, and returns a particular value from the attribute function range. The attribute function sets UA , DA , OPA , and $INFOA$ are the set of attribute functions associated with users, devices, operations, and information, respectively. Context state attribute functions set (CSA) describes the context characteristics of the current instance of time, which include both environment-related characteristics, for example, "day", "time", and "weather condition", or the system-related attributes, for instance, "the operating system", "the security condition", and the like. Actor's sessions' attributes are classified into two categories, **Users' session attribute functions set (USA)** and **devices' session attribute functions set (DSA)**. USA and DSA are the sets of attributes associated with the sessions created by users and by devices, respectively. Some of the users' session attribute functions are inherited from the session's unique user creator, denoted as inherited user attribute functions (IUA). If a session s_u inherited a user attribute ua from its user $user(s_u)$, then it is required that $ua(s_u) = ua(user(s_u))$. For instance, suppose we have a

user attribute function *age*, which returns the user age, and suppose that for *Bob*, $age(Bob) = 10$. Then, if *Bob* created a session s_x , and if the session s_x inherited *Bob*'s age attribute, we will have $age(s_x) = age(Bob) = 10$. How the inheritance process happens is outside the scope of this operational model and can be considered part of an administrative model. Similarly, some of the devices' session attribute functions are inherited from the session's unique device creator attributes, denoted as **inherited device attribute functions (IDA)**. If a session s_d inherited a device attribute da from its unique device creator $device(s_d)$, then it is required that $da(s_d) = da(device(s_d))$. On the other hand, some of the users' session attribute functions and the devices' session attribute functions are unique and not inherited from the session's corresponding creator, denoted as S_uA and S_dA , respectively. These unique attributes may include session-related attributes, for instance, the connection type, the session time out, and so on. An attribute range is composed of a finite set of atomic values. Each attribute function *att* has a range $Range(att)$, where, $Range(att)$ represents the range of values which the attribute function *att* can be evaluated to. When an atomic attribute is evaluated, it will return exactly one value from its range, while when a set attribute is evaluated, it will return a subset of values from its range. Over time, all attributes may change. However, some attributes are "relatively" static, where they continue to evaluate to the same value over a long period of time. Setting and changing the values of static attributes may require administrator intervention, for instance, the user job title. On the other hand, dynamic attributes keep changing due to various events, such as the user's location and the device's usage status. Dynamic attributes are automatically set by sensors deployed in the smart system. Attribute functions are formally defined in Table 3 in Appendix A.

Guardian Attribute Functions. These are special kinds of attribute functions. We have two types of guardian attribute sets: **user guardian attribute functions set (UGA)** and **device guardian attribute functions set (DGA)**. Similar to the attribute functions, each guardian attribute function *ga* has a finite range of atomic values $Range(ga)$. Each guardian attribute function is either an atomic value function or a set value function. The system administrator assigns different user and device guardian attributes to different users and devices, respectively. The permission constraints use Guardian attributes to prevent unwanted access, as explained in the following constraints paragraph. Guardian attribute functions are formally defined in Table 4 in Appendix A.

Authorization Functions. Attribute authorization functions are Boolean functions. We have four different cases: (a) Users to devices attributes authorization function ($Authorization_{U-D}$), evaluated when a user session tries to access an operation on a device; (b) Devices to devices attributes authorization function ($Authorization_{D-D}$), evaluated when a device session tries to access an operation on a device; (c) Users to information attributes authorization function ($Authorization_{U-INFO}$) is evaluated when a user session tries to access an operation on information generated by or stored on a device; (d) Devices to information attributes authorization function ($Authorization_{D-INFO}$), evaluated when a device session tries to access an operation on information generated by or stored on a device. Attribute authorization functions are formally defined in Table 4 in Appendix A.

Constraints. Constraints are invariants that must be maintained. We have three types: constraints on actor attributes, constraints on actor's sessions' attributes, and permission constraints.

Actor's attributes' constraints are of two types: **users attributes constraints (UAConstraint)** and **devices attributes constraints (DAConstraint)**. They encompass the static separation of duty concept [63]. They impose restrictions on users' attributes and devices' attributes. For instance, in the case of users attributes constraints, if a specific attribute value is assigned to a user, the user is prohibited from being assigned to another attribute value. In defining this type of constraint, we use the user attribute pairs set UAP as shown in Table 5 in Appendix A. UAP is a set of pairs. Each pair is a pair of a user attribute function and a user attribute function value. For

each pair (ua, v) in a UAP set, ua is an attribute function in the user attribute functions set, while v is a value in the ua attribute function range of values. This is formally defined as following: $UAP = \{(ua, v) \mid ua \in UA \wedge v \in Range(ua)\}$. Users attributes constraints are defined as a one-to-many user attribute pairs to user attribute pairs constraints relation. For example, consider the following user attribute constraint:

$$UAC_{ac_i} = \{uac_i\}$$

$$uac_i = ((Relationship, kid), \{(HighControl, True)\})$$

This constraint implies that for any user u_x with an attribute value pair $(Relationship, kid)$ (i.e., $Relationship(u_x) = kid$), u_x cannot have the attribute function value $(HighControl, True)$. In other words, u_x cannot be assigned to the attribute $HighControl$ with the value $True$ (i.e., $HighControl(u_x) \neq True$). Devices attributes constraints are defined similarly to users attributes constraints.

Session's attributes' constraints are of two types: **user's sessions attributes constraints** ($USAC_{ac_i}$) and **device's sessions attributes constraints** ($DSAC_{ac_i}$). They encompass the dynamic separation of duty concept [63]. They impose restrictions on the user's sessions' attributes and device's sessions' attributes. For instance, in the case of user's sessions attributes constraints, if a specific attribute value is assigned to a user's session, the session is prohibited from being assigned to another attribute value. In defining this type of constraint, we use the user's session's attribute pairs set $USAP$ as shown in Table 5 in Appendix A. $USAP$ is a set of pairs. Each pair is a pair of a user's session attribute function and a user's session attribute function value. For each pair (usa, v) in a $USAP$ set, usa is an attribute function in the users' sessions attribute functions set, while v is a value in the usa attribute function range of values. This is formally defined as follows: $USAP = \{(usa, v) \mid usa \in USA \wedge v \in Range(usa)\}$. User's sessions attributes constraints are defined as a one-to-many user's session's attribute pairs to user's session's attribute pairs constraints relation, e.g., the following:

$$USAC_{ac_i} = \{usac_i\}$$

$$usac_i = ((AdminRole, True), \{(OperationalRole, True)\})$$

Here, it is permissible for users' sessions to be assigned to different attribute values, which do not constitute a conflict of interest when assigned independently to different sessions that belong to the same user but produce policy concerns when allowed to be assigned together in the same session. For example, the above constraint implies that for any user's session s_x with an attribute value pair $(AdminRole, True)$ (i.e., $AdminRole(s_x) = True$), s_x cannot have the attribute function value pair $(OperationalRole, True)$ (i.e., $OperationalRole(s_x) \neq True$). However, there may exist another session s_y , where $user(s_x) = user(s_y) = u_x$ and $OperationalRole(s_y) = True$. The device's sessions attributes constraints are defined similarly to the user's sessions attributes constraints. Enforcing the actor's attributes' and the session's attributes' constraints is outside this operational model's scope and is considered part of the administration model.

Permission constraints prevent specific actors from accessing specific operations on specific objects. By assigning different actors to different guardian attributes, permission constraints are then defined as a many-to-many permissions to guardian attributes constraints relation. These constraints enable $UCON_{IoT}$ to limit the set of permissions available for each actor. We have two types of permission constraints, users' permissions constraints and devices' permissions constraints. In defining users' permissions constraints, we use the user guardian attribute pairs set $UGAP$ as shown in Table 5 in Appendix A. $UGAP$ is a set of pairs. Each pair is a pair of a user guardian attribute function and a user guardian attribute function value. For each pair (uga, v) in a $UGAP$ set, uga is an attribute function in the user guardian attribute functions set, while v is a value in the uga attribute function range of values. This is formally defined as follows:

$UGAP = \{(uga, v) \mid uga \in UGA \wedge v \in Range(uga)\}$. Users' permissions constraints are defined as many-to-many subset of permissions to set of user guardian attribute pairs. Having a users' permission constraint $upc_i \in UPConstraint$, where $upc_i = (\{(d_k, op_l)\}, \{(uga_x, v_y)\})$ implies that a user u_i , which is assigned to the user guarding attribute uga_x with the value v_y ($uga_x(u_i) = v_y$) cannot access the operation op_l on the device d_k , e.g., as follows:

$$UPConstraint = \{upc_i\}$$

$$upc_i = (\{(Unlock, SmartCar), (TurnOn, Oven)\}, \{(Kids, true)\})$$

This constraint implies that for any user u_x with a guardian attribute value pair $(Kids, true)$ (i.e., $Kids(u_x) = true$), user u_x cannot have access to the following permissions: (a) Unlock the smart car ($Unlock, SmartCar$) and (b) Turn on the oven ($TurnOn, Oven$). This prevention is enforced as part of the check access predicate. Devices' permissions constraints are defined similarly to users' permission constraints. Permission constraints are crucial since they prescribe limits on the set of permissions available for each actor. They prevent specific actors from accessing specific permissions even if they are inadvertently or maliciously authorized to access them by the authorization function.

Check Permission Constraints Function. Depending on the four combinations of actors and targets, we have four check permission constraints functions. These functions are evaluated as part of the check access predicate evaluation given below. Check permission constraints ensure that the requesting actor is not prohibited from accessing the requested permission by one of the permission constraints. They are formally defined in Table 5 in Appendix A.

Check Access Predicate Function. Corresponding to the four combinations of actors and targets, we have four check access predicates. These predicates are formally described in Table 6 in Appendix A. When a user's session s_{u_i} attempts an operation op_k on a device d_j , the ADE engine evaluates the $CheckAccess(s_{u_i}, op_k, d_j, current)$ predicate. When a device's session s_{d_i} attempts an operation op_k on a device d_j , the ADE engine evaluates the $CheckAccess(s_{d_i}, op_k, d_j, current)$ predicate. Moreover, when a user's session s_{u_i} attempts an operation op_k on information $info_l$ on a device d_j , the ADE engine evaluates the $CheckAccess(s_{u_i}, op_k, d_j, info_l, current)$ predicate. Finally, when a device's session s_{d_i} attempts an operation op_k on information $info_l$ on a device d_j , the ADE engine evaluates the $CheckAccess(s_{d_i}, op_k, d_j, info_l, current)$ predicate. The running check access predicate perform three checks, as follows: (i) If the target object is a device, it checks if the requested operation is actually allowed on the target device, i.e., $(op_k \in ops(d_j))$. On the other hand, if the requested target object is information, it checks if the requested information belongs to the requested device and if the requested operation is allowed to be performed on the target information, i.e., $(op_k \in ops(info_l) \wedge info_l \in info(d_j))$; (ii) If the requesting actor is a user and the target object is a device, it checks the **users-to-devices (U-D)** attributes authorization function. If the requesting actor is a user and the target object is information, it checks the **users-to-information (U-INFO)** attributes authorization function. Moreover, if the requesting actor is a device and the target object is a device, it checks the **devices-to-devices (D-D)** attributes authorization function. Finally, if the requesting actor is a device and the target object is information, it checks the **devices-to-information (D-INFO)** attributes authorization function; (iii) Depending on the actor and target combination, it checks the relevant permission constraint function, to ensure that the requesting actor is not prohibited to this action by one of the permission constraints.

Attributes Update Functions. $UCON_{IoT}$ supports mutable attributes. It enables updating different attribute values before, during, and after access enforcement. We have four types of before execution update functions depending on the four different combinations of the actors and targets. $PreUpdate_{U-D}$, $PreUpdate_{D-D}$, $PreUpdate_{U-INFO}$, and $PreUpdate_{D-INFO}$ functions are

executed after the requested access is authorized and before the requested access is enforced if the actor is a user and the target is a device, if the actor is a device and the target is a device, if the actor is a user and the target is information, and if the actor is a device and the target is information, respectively. Similarly, $U\text{CON}_{IoT}$ supports four types of during execution update functions depending on the four different combinations of the actors and targets ($OnUpdate_{U-D}$, $OnUpdate_{D-D}$, $OnUpdate_{U-INFO}$, and $OnUpdate_{D-INFO}$). These update functions are executed after the requested access is authorized and while the requested access is enforced. Furthermore, $U\text{CON}_{IoT}$ supports four post execution update functions depending on the four different combinations of the actors and targets, $PostUpdate_{U-D}$, $PostUpdate_{D-D}$, $PostUpdate_{U-INFO}$, and $PostUpdate_{D-INFO}$. These update functions are executed after the requested access is executed. Table 7 in Appendix A provide a formal specification language for the attributes update functions.

5 Use Case Scenarios

5.1 Users as the Actors

This section introduces a smart health care system use case as shown in Figure 8. It illustrates user-to-device and user-to-information access configuration in $U\text{CON}_{IoT}$. In this use case, Alice is the patient, and John is Alice's doctor, who monitors her remotely after recent surgery. Here, we have three users, Alice, Bob, and John. We have one user attribute, *Relationship*, which defines the user relationship in the smart system. We have $Relationship(Alice) = Patient$, $Relationship(Bob) = PatientSpouse$, and $Relationship(John) = Doctor$, indicating that Alice is the patient, Bob is the patient spouse, and John is the doctor, receptively. Moreover, we have two devices, a smartwatch, and a smart weight scale. We have one device attribute, *OwnerD*, which identifies the devices' owners. While the smartwatch owner is the patient (Alice), the smart weigh scale owners are both the patient and the patient spouse (Alice and John). We have eight information objects. In the smartwatch device, we have four information objects (heart rate, temperature, blood pressure, and location). In the smart weight scale device, we have four information objects (Alice's weight, Alice's BMI, Bob's weight, and Bob's BMI). Furthermore, we have two information attributes. *OwnerInfo* attribute identifies the information object's owner and *SensitiveInfo* which indicates the sensitivity level of the information object. We have one context state attribute *Emergency*, which indicates whether there is an emergency regarding a health condition of Alice or not. How the context condition attribute is set and updated is outside the scope of this policy model. We have five device operations; in the smartwatch, we have activate and deactivate operations. We have activate, deactivate, and reset operations on the smart weight scale. Moreover, each information object has a read operation that can be performed on it; hence, a total of eight information operations belong to different information objects. We have four operation attributes. The control operation attribute (*ControlOp*) evaluates to true if the operation is a control operation. Activating and deactivating the smartwatch and the smart weight scale are classified as control operations. The read operation attribute (*ReadOp*) indicates whether the operation is a read operation or not. All reading operations on different information objects are classified as reading operations. Reset operation (*ResetOp*) indicates whether the operation is a reset operation or not. Reset the smart weight scale is a reset operation. Finally, the reset counter operation attribute (*ResetCounter*) indicates how many times the reset operation can be performed in one day (how many times we can reset the smart weight scale). How this counter is updated at the beginning of each day is outside the scope of this policy model. The reset counter operation attribute is a mutable attribute. It is written in green in Figure 8. Finally, we have two authorization functions: (a) User-to-device authorization function ($Authorization_{U-D}$), which governs users to devices access authorization; (b) User-to-information authorization function ($Authorization_{U-INFO}$),

```

U = {Alice, Bob, John}, UA = {Relationship}, Relationship : u : U → {Patient, PatientSpouse, Doctor}
Relationship(Alice) = Patient, Relationship(Bob) = PatientSpouse, Relationship(John) = Doctor

D = {SmartWatch, SmartWeightScale}
DA = {OwnerD}
OwnerD : d : D → 2^{Patient, PatientSpouse, Doctor}
OwnerD(SmartWatch) = {Patient}, OwnerD(SmartWeightScale) = {Patient, PatientSpouse}

INFO = info(SmartWatch) ∪ info(SmartWeightScale)
info(SmartWatch) = {HeartRate, Temperature, BloodPressure, location},
info(SmartWeightScale) = {AliceWeight, AliceBMI, BobWeight, BobBMI}
INFOA = {OwnerInfo, SensetiveInfo}
OwnerInfo : info : INFO → {Patient, PatientSpouse, Doctor}, SensetiveInfo : info : INFO → {x|x is a numerical number}
OwnerInfo(HeartRate) = OwnerInfo(Temperature) = OwnerInfo(BloodPressure) = OwnerInfo(location) = OwnerInfo(AliceWeight) =
OwnerInfo(AliceBMI) = {Patient}
OwnerInfo(BobWeight) = OwnerInfo(BobBMI) = {PatientSpouse}
SensetiveInfo(HeartRate) = SensetiveInfo(Temperature) = SensetiveInfo(BloodPressure) = 4, SensetiveInfo(location) = 8
All other values are undefined.

CS = {Current}
CSA = {Emergency}
Emergency : current : CS → {True, False}

OP = OP_d ∪ OP_info
OP_d = OP_SmartWatch ∪ OP_SmartWeightScale
OP_SmartWatch = {ActivateSmartWatch, DeactivateSmartWatch},
OP_SmartWeightScale = {ActivateSmartWeightScale, DeactivateSmartWeightScale, ResetSmartWeightScale}
OP_info = OP_HeartRate ∪ OP_Temperature ∪ OP_BloodPressure ∪ OP_location ∪ OP_AliceWeight ∪ OP_AliceBMI ∪ OP_BobWeight ∪ OP_BobBMI
OP_HeartRate = {ReadHeartRate}, OP_Temperature = {ReadTemperature}, OP_BloodPressure = {ReadBloodPressure}, OP_location = {Readlocation},
OP_AliceWeight = {ReadAliceWeight}, OP_AliceBMI = {ReadAliceBMI}, OP_BobWeight = {ReadBobWeight}, OP_BobBMI = {ReadBobBMI}
OPA = {ControlOp, ReadOp, ResetOp, ResetCounter}
ControlOp : op : OP → {True, False}, ResetOp : op : OP → {True, False}
ReadOp : op : OP → {True, False}, ResetCounter : op : OP → {x|x is a numerical number}
ControlOp(ActivateSmartWatch) = ControlOp(DeactivateSmartWatch) = ControlOp(ActivateSmartWeightScale) =
ControlOp(DeactivateSmartWeightScale) = True
ReadOp(ReadHeartRate) = ReadOp(ReadTemperature) = ReadOp(ReadBloodPressure) = ReadOp(Readlocation) =
ReadOp(ReadAliceWeight) = ReadOp(ReadAliceBMI) = ReadOp(ReadBobWeight) = ReadOp(ReadBobBMI) = True
ResetOp(ResetSmartWeightScale) = True
ResetCounter(ResetSmartWeightScale) = 1. This attribute is set to one at the beginning of each day.
All other values are undefined.

Authorization_U-p(s_u : S_u, op : OP, d : D, current : ES) ≡ Relationship(s_u) ∈ OwnerD(d) ∧ ControlOp(op) = True ∨
Relationship(s_u) ∈ OwnerD(d) ∧ ResetOp(op) = True ∧ ResetCounter(op) = 1

Authorization_U-INFO(s_u : S_u, op : OP, d : D, info : INFO, current : ES) ≡
(Relationship(s_u) = OwnerInfo(info) ∧ ReadOp(info) = True ∨
(Relationship(s_u) = Doctor ∧ OwnerInfo(info) = Patient ∧ SensetiveInfo(info) ≤ 5 ∧ ReadOp(op) = True ∨
(Relationship(s_u) = Doctor ∧ OwnerInfo(info) = Patient ∧ SensetiveInfo(info) > 5 ∧ ReadOp(op) = True ∧ Emergency(current) = True

PostUpdate_U-D_ResetSmartWeightScale(s_u : S_u, d : D, current : EC) ≡ True → ResetCounter(op) = ResetCounter(op) - 1

```

Fig. 8. Smart health care use case configuration (user to device and user to information policy).

which governs users to information access authorization. The objectives of the user-to-device authorization function are: (a) Allow devices' owners to be able to activate and deactivate the devices that they own (the smart weight scale can be activated and deactivated by Alice and Bob, while the smartwatch can be activated and deactivated by Alice only); (b) Allow Alice and Bob to reset the smart weight scale one time per day only. This is achieved by first having one reset counter attribute and associating it with the smart weight scale's reset operation with a value that is set to one at the beginning of each day. Second, by defining a user-to-device post-execution update function that decreases the reset counter attribute value associated with the smart weight scale's reset operation when this operation is executed. The goals of the device-to-information authorization function are: (a) Allow owners of information objects to be able to read their information (Alice can read her information generated by the smartwatch and the smart weight scale, and Bob can read his information generated by the smart weight scale); (b) Allow the doctor to read his patient's information without context conditions only if the sensitivity levels of this information are below 5 (John can read all of Alice's information except her location); (c) Allow the doctor to read his patient's information with sensitivity levels above 5 only during emergencies (John can read Alice's location in emergencies).

```

D = {TemperatureMonitor, thermostat, SmokeDetector, CarbonMonoxideDetector, FrontDoorlock, BackDoorlock, Window1, Window2, Window3, Window4}
DA = {AlarmD, VentilationD, MonitoringD, MonitoredD, IncreasedByMonitor, DecreaseByMonitor}
AlarmD : d : D → {True, False}, VentilationD : d : D → {True, False} MonitoringD : d : D → {True, False}
MonitoredD : d : D → {True, False},
IncreasedByMonitor : d : D → {x|x is 0 or a positive integer}, DecreaseByMonitor : d : D → {x|x is 0 or a positive integer}
MonitoringD(TemperatureMonitor) = True, MonitoredD(thermostat) = True
AlarmD(SmokeDetector) = AlarmD(CarbonMonoxideDetector) = True
VentilationD(FrontDoorlock) = VentilationD(BackDoorlock) = VentilationD(Window1) = VentilationD(Window2) =
    VentilationD(Window3) = VentilationD(Window4) = True,
IncreasedByMonitor(thermostat) = DecreaseByMonitor(thermostat) = 0, These two attributes are set to zero at the beginning of each day.
All other values are undefined.

INFO = info(thermostat) = {CurrentTemperature}, INFOA = {} CS = {Current}, CSA = {}

OP = OP_d ∪ OP_info
OP_d = OP_TemperatureMonitor ∪ OP_thermostat ∪ OP_SmokeDetector ∪ OP_CarbonMonoxideDetector ∪ OP_FrontDoorlock ∪ OP_BackDoorlock ∪ OP_Window1 ∪
    OP_Window2 ∪ OP_Window3 ∪ OP_Window4
OP_TemperatureMonitor = {ActivateTemperatureMonitor, DeactivateTemperatureMonitor}, OP_thermostat = {DecreaseTemperature, IncreaseTemperature}
OP_FrontDoorlock = {LockFrontDoorlock, UnlockFrontDoorlock}, OP_BackDoorlock = {OpenBackDoorlock, CloseBackDoorlock},
OP_Window1 = {OpenWindow1, CloseWindow1}, OP_Window2 = {OpenWindow2, CloseWindow2},
OP_Window3 = {OpenWindow3, CloseWindow3}, OP_Window4 = {OpenWindow4, CloseWindow4}
OP_info = OP_CurrentTemperature = {ReadCurrentTemperature}
OPA = {VentilationOp, ReadingOp, MonitoringOp}
VentilationOp : op : OP → {True, False}, ReadingOp : op : OP → {True, False}, MonitoringOp : op : OP → {True, False},
VentilationOp(UnlockFrontDoorlock) = VentilationOp(OpenBackDoorlock) = VentilationOp(OpenWindow1) =
    VentilationOp(OpenWindow2) = VentilationOp(OpenWindow3) = VentilationOp(OpenWindow4) = True
MonitoringOp(DecreaseTemperature) = MonitoringOp(IncreaseTemperature) = True, ReadingOperation(ReadCurrentTemperature) = True
All other values are undefined.

Authorizationp-p(sd : Sd, op : OP, d : D, current : CS) ≡ (AlarmD(sd) = True ∧ VentilationD(d) = True ∧ VentilationOp(op) = True) ∨
(MonitoringD(sd) = True ∧ MonitoredD(d) = True ∧ MonitoringOperation(d) = True)

Authorizationp-info(sd : Sd, op : OP, d : D, info : INFO, current : CS) ≡
(MonitoringD(sd) = True ∧ MonitoredD(d) = True ∧ ReadingOperation(op) = True)

PostUpdatep-D-DeceaseTemperature(sd : Sd, d : D, current : CS) ≡
    MonitoringD(sd) = True → IncreasedByMonitor(d) = IncreasedByMonitor(d) + 1
PostUpdatep-D-DeceaseTemperature(sd : Sd, d : D, current : CS) ≡
    MonitoringD(sd) = True → DecreaseByMonitor(d) = DecreaseByMonitor(d) + 1

```

Fig. 9. Smart manufacturing use case configuration (device-to-device and device-to-information-policy).

5.2 Devices as the Actors

This section introduces a smart manufacturing use case as shown in Figure 9. It illustrates device-to-device and device-to-information access configuration in $UCON_{IoT}$. In this use case, we have ten devices: a temperature monitor, thermostat, smoke detector, carbon monoxide detector, front door lock, back door lock, and four windows. Moreover, we have different device attributes. The alarm device attribute (*AlarmD*) evaluates to true-or-false to indicate whether the device is an alarm device or not. The smoke detector and the carbon monoxide detector are classified as alarm devices. The ventilation device attribute (*VentilationD*) evaluates to true-or-false to indicate whether the device is a ventilation device or not. The front door lock, the back door lock, and all the system windows are classified as ventilation devices. The monitoring device attribute (*MonitoringD*) indicates whether the device is a monitoring device or not. The temperature monitor device is a monitoring device. Finally, the monitored device attribute (*MonitoredD*) evaluates to true for the devices classified as monitored devices; the thermostat device in our use case is a monitored device. Also, we have one information object, *CurrentTemperature*, which is stored by the thermostat device. We have sixteen operations belonging to different devices and information objects. Furthermore, we have three operation attributes. Ventilation operation attribute (*VentilationOp*) evaluates to true for ventilation operations. Opening the front door lock, the back door lock, and the system's windows are classified as ventilation operations. The monitoring operation attribute (*MonitoringOp*) evaluates to true for monitoring operations. Increasing and decreasing the temperature of the thermostat device are categorized as monitoring operations. Furthermore, the reading operation attribute (*ReadingOp*) indicates whether the operation is a reading operation or not. Reading the temperature information on the thermostat device is categorized as a reading

operation. Finally, we have two authorization functions: (a) Devices-to-devices authorization function ($Authorization_{D-D}$), which governs devices to devices access; (b) Devices-to-information authorization function ($Authorization_{D-INFO}$), which governs devices-to-information-access. The objectives of the devices-to-devices authorization function are: (a) Allow alarm devices (the smoke detector and the carbon monoxide detector) to perform ventilation operations (unlock the system's doors and open the system's windows) on ventilation devices (the front door lock, the back door lock, and the system's windows); (b) Allow monitoring devices (the temperature monitor) to perform monitoring operations (increasing and decreasing the temperature) on monitored devices (the thermostat). The goal of the devices to information authorization function is to allow monitoring devices (the temperature monitor) to perform reading operations (read the current temperature) on the information of its monitored devices (the current temperature information on the thermostat device). Moreover, we have two mutable device attributes *IncreasedByMonitor* and *DecreaseByMonitor*, written in green in Figure 9. These counters indicate how many times the thermostat temperature was increased or decreased, respectively. Accordingly, we have two post-execution update functions.

6 Proof-of-Concept Implementation

In this section, we utilize **Amazon Web Services (AWS)** IoT service [1] to implement the use case of Section 5.2 and Figure 9 as it is one of the most mature IoT platforms available on the market. Ahmed et al. [2, 3] highlight adjustments made by **Cloud Service Providers (CSPs)**, including AWS, to adapt access control methods for IoT security. However, these approaches face limitations such as restricted policy administration support, reliance on proprietary policy languages leading to vendor lock-in, and limited expressiveness in authorization constraint specification. To address these challenges, Ahmed et al. [3] proposed using stateless functions (lambda functions) to enhance AWS IoT and Greengrass's native access control mechanism, presenting five deployment models with differing access control logic placement (cloud or edge) and policy evaluation attributes management. Our implementation aligns with one of their deployment models, which is recognized as the CDM2 deployment model, where access control logic is cloud-hosted, and necessary attributes are stored and retrieved from a cloud repository. Our proof of concept implementation aims to demonstrate the applicability of our proposed $UCON_{IoT}$ access control policy model presented in Section 4. This demonstration is facilitated by our ZTA-IoT architecture, detailed in Section 3.1 and depicted in Figure 3, utilizing commercially available AWS IoT service. An AWS IoT Core instance was set up to manage an IoT ecosystem with ten devices. Each device was registered within AWS IoT Core through the console. AWS IoT's "one-click certificate" functionality was utilized to obtain an x.509 certificate, a public key, and a private key for each device to establish secure connections between IoT devices and AWS IoT. An AWS IoT authorization policy was attached to each certificate, enabling IoT devices communication with an application on an EC2 instance via an MQTT broker. Ten MQTT topics were established to facilitate communication, each IoT device configured as a publisher, and the EC2 application as a subscriber. Additionally, ten more MQTT topics were created, configuring each IoT device as a subscriber and the EC2 application as a publisher. An EC2 instance was set up to act as an intermediary between AWS IoT Core and lambda function after installing AWS IoT SDK on it. EC2 can host the logic necessary to communicate with both services. A Python script application was created on the EC2 instance to communicate with IoT devices using MQTT connections and to initiate the lambda function. The lambda function was developed as a policy engine, leveraging Python and the Boto3 library to interact with the EC2 application and the DynamoDB tables. IAM roles were configured for the lambda function, providing the required permissions for DynamoDB table access and interaction with the EC2 application. To enforce our use case, several DynamoDB tables were created to store

various attributes as follows: The Devices table contains different devices and their unique IDs, the Device Attributes table contains the attribute values of different devices, the Information table holds different information objects along with their associated information IDs, and the devices they belong to. In our specific use case, only one information object (*CurrentTemperature*) exists, resulting in a single row in this table. The Operation table stores different operations, their unique IDs, and their associations with devices or information, depending on whether the operation pertains to information or device. The Operation attributes table stores the attribute values of different operations. The Authorization table stores authorization policies, along with their associated IDs and types, which can be either **device-to-device (D-D)** or **device-to-information (D-INFO)** authorization policies. In our use case, we have two authorization policies, resulting in two rows within this table. The ADE engine, discussed in Section 4.1, determines access approval or denial based on its implemented algorithm, utilizing the predefined policy model of the AE and score and threshold values calculated by the score engine. Since the score engine design is left as a future direction, in this implementation, it's assumed that the score consistently surpasses the threshold function. The AE predefined policy model applies the smart manufacturing use case outlined in Section 5.2 and Figure 9. The $ADE_{UCON_{IoT}}$ algorithm orchestrates the sequence of actions in the lambda Python code, while the AE logical model defines system components, attribute update functions, and authorization functions utilized by the $ADE_{UCON_{IoT}}$ algorithm for access decision-making. The code operates via a multi-threaded lambda, as depicted in Figure 10. Consider the following scenario, the temperature monitor IoT device wants to increase the thermostat temperature. It publishes the request to the designated MQTT topic within the AWS IoT Core environment. The EC2 application which is subscribed to the relevant MQTT topic, receives this request. Upon message reception, the application will trigger the lambda function, providing it with IoT request details. The lambda function retrieves necessary attributes from DynamoDB, tailored to the request type (D-D or D-INFO). If the request is a D-D request, it retrieve the attributes of the requesting device, the requested device, and the requested operation. Since in this scenario the request is a D-INFO request, the lambda function retrieves attributes related to the requesting device (*TemperatureMonitor*), the requested device (*thermostat*), the requested information on the requested device (*CurrentTemperature*), and the attributes of the requested operation on the requested information (*IncreaseTemperature*) from the DynamoDB. Also, if the request is a D-D request, the lambda function will retrieve the D-D type authorization function from the authorization table. However, in this scenario, it is a D-INFO type authorization, accordingly, the lambda function will retrieve the D-INFO type authorization function from the authorization table. The lambda function will then parse the retrieved authorization function, if the retrieved attributes satisfy the authorization function, the access is granted; if not, the access is denied. The lambda function will then utilize the Boto3 library to send the response back to the EC2 application. The EC2 application then communicates the result back to the temperature monitor via the MQTT topic.

We conducted a comprehensive performance analysis on our policy model implementation. This analysis included measuring the average access request transmission time from IoT devices to the EC2 application, the average Lambda processing time, and the average DynamoDB query retrieval time under various conditions. Seven different scenarios with varying loads of concurrent access requests were tested, each executed one hundred times to calculate averages. Additionally, the number of retrieved attributes from the database was recorded for each load of requests. In our implemented use case, consisting of only ten devices, we assessed scalability by simulating varying numbers of concurrent requests. By replicating IoT device instances, creating two, three, four, and five instances of each device, we generated scenarios with 20, 30, 40, and 50 requests from diverse devices simultaneously. It's noteworthy that instances of identical devices share the same device

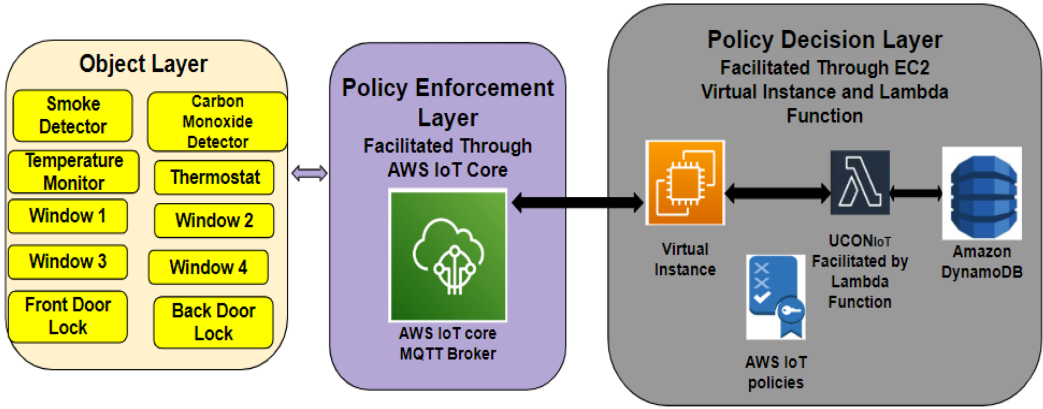


Fig. 10. Smart manufacturing use case implementation using AWS IoT.

Table 1. AWS IoT Implementation Data Processing Performance Summary

Number of concurrent requests	Average Access Request Transmission Time from IoT Device to EC2 Application (Milliseconds)	Average Lambda Processing Time (Milliseconds)	Number of Retrieved Attributes from DynamoDB	Average DynamoDB Query retrieval Time (Milliseconds)
1	1544.49	169.54	15	122.06
5	1833.26	219.06	75	198.08
10	2559.78	329.29	150	312.76
20	3488.12	358.45	300	339.94
30	4760.78	308.91	450	287.9
40	5879.35	226.08	600	206.94
50	6993.65	251.17	750	230.88

ID and utilize the same MQTT topic when communicating with the AWS IoT Core. The results are depicted in Table 1. From the results, and from Figure 10, we can conclude that our proposed UCON_{IoT} policy model demonstrates functionality and applicability using commercially available technology, which aligns with the primary objective of this proof of concept implementation. The transmission time for access requests from IoT devices to the EC2 application increases significantly with higher concurrent access request loads. This is mainly due to instances of the same devices sharing the same MQTT topic. Additionally, our implementation utilizes a single EC2 application to handle all requests. Furthermore, we observed that the average lambda processing time is not directly proportional to the number of concurrent requests and generally remains low. This is attributed to our implementation of a multi-threaded lambda. Similarly, the DynamoDB query retrieval time is generally low and not directly proportional to the number of concurrent requests. This is due to two factors, our choice of a multi-threaded lambda and our decision to use DynamoDB, known as a fully managed NoSQL database service that provides fast and predictable performance with seamless scalability.

7 Related Work

In IoT systems, security and privacy present unique challenges [13, 29, 36]. Most researchers consider providing appropriate access control for IoT as one of the most important challenges. Multiple approaches have been proposed for IoT access control. Some of them are based on RBAC [35, 59] (e.g., [8], [15], [16], [19], [27], and [75]). However, capturing constantly changing dynamic

attributes in RBAC-based models may result in role explosion [7, 9, 43]. Other solutions are based on ABAC [40, 41] (e.g., [9], [12], [17], and [20]). However, as shown in [7], ABAC-based models are not capable of prescribing limits on subject permissions. Furthermore, both ABAC- and RBAC-based models do not support continuous authorization control. Some blockchain, cryptographic mechanisms, capabilities, access control lists, and policy-based solutions have been introduced for enforcing access control policies in IoT [23, 34, 70]. Several other access control models for IoT have been developed; Ouaddah et al. [47], Qiu et al. [52], and Ravidas et al. [54] provided surveys on them.

Recently, Ameer et al. motivated the need to incorporate ZT principles in designing, enforcing, and implementing access control models for IoT systems [10]. In the literature, few solutions converging the ZT paradigm in IoT systems have been suggested. These solutions take a disjointed approach, focusing on a specific security function or layer of the IoT network. A holistic approach addressing IoT security concerns across multiple layers of the network and multiple security functions is still missing. In the following, we discuss different proposed solutions in the literature (to our best knowledge) that integrate the ZT paradigm in IoT systems. We compare these solutions with our proposed work against the following criteria: (1) *Comprehensive Approach*. This characteristic determines whether the proposed solution considers different types of actors and targets across multiple layers of the IoT network; (2) *Formal Policy Model*. This characteristic determines whether the proposed solution provides a formal authorization policy model. We believe that providing a formal policy model is critical so that there is a precise and rigorous specification of the intended behavior; (3) *Attribute Aware Authorization*. This characteristic determines whether the proposed solution supports an authorization model capable of capturing different actors', targets', and environmental attributes; (4) *Continuous Authorization*. This characteristic determines whether the proposed solution supports an authorization model which is dynamic enough to allow continuous verification of access control authorization policies to ensure ongoing control for relatively long-lived operations or immediate revocation; (5) *Score-Based Authorization*. This characteristic determines whether the proposed solution supports an authorization model that is dynamic enough to take into account the level of confidence (score) associated with different access requests and to accommodate subjective information and uncertainty rather than deciding on access based on static access control policies only; (6) *Use Case Demonstration*. This characteristic determines whether the proposed solution is demonstrated using use case scenarios to show its applicability in real world situations; (7) *Implementation*. This characteristic determines whether the proposed solution is demonstrated through a proof of concept or experimental implementation. This is critical to show that the proposed solution is credible using commercially available technology with necessary enhancements.

Our comparison is comprehensively presented in Table 2. From Table 2, we can notice that our proposed solution supports all the comparison criteria characteristics. Most importantly, except for our solution, no solution took a holistic approach to address IoT systems' interactions across multiple layers of the IoT network.

Samandiego and Deters [57] introduced blockchain-based middleware to handle IoT network infrastructure and transaction management in a zero-trust manner. However, as shown in Table 2, their architecture is a high-level logical architecture. They didn't consider the interactions between different components in IoT systems. Moreover, they described their authorization model in a high-level manner without providing a formal policy model or a policy specification. Except for users' identity information, their model cannot capture other attributes like requested target attributes and environment attributes. Additionally, their framework doesn't support score-based authorization. Finally, they didn't demonstrate their framework with use case scenarios. Dhar and Bose [30] proposed a ZT blockchain High-level network implementation architecture for IoT. However,

Table 2. A Review of Related Work on ZT IoT Access Control Against Criteria Adapted from [10]

Related Work	Comprehensive Approach	Formal Policy Model	Attribute Aware Authorization	Continuous Authorization	Score-based Authorization	Use Case Demonstration	Implementation
Our contribution	Yes	Yes	Yes	Yes	Yes	Yes	Yes
[57]	No	No	No	No	No	No	Yes
[30]	No	No	NA	NA	NA	No	No
[25]	No	No	NA	NA	NA	No	No
[73]	No	No	NA	NA	NA	No	No
[24]	No	No	Yes	Yes	Yes	No	Yes
[68]	No	No	NA	NA	NA	Yes	Yes
[71]	No	Yes	Yes	Yes	Yes	Yes	No
[31]	No	Yes	Yes	Yes	Yes	Yes	Yes
[45]	No	Yes	No	No	Yes	Yes	Yes

their architecture only considers devices-to-devices, devices-to-users, and devices-to-applications interactions. They mainly focused on providing an implementation architecture; hence, they didn't give an authorization policy model that governs different types of access requests. Accordingly, the third, fourth, and fifth characteristics in Table 2 are not applicable. Additionally, they didn't provide a use case demonstration for their model. Finally, they didn't implement or test their architecture. Chen et al. [25] suggested using blockchain to enhance data interaction security in all links of the power network in a ZT approach. However, their model is developed to secure operators (users) to data access in electrical power supply chain networks in specific. They proposed an enforcement architecture model without considering providing an authorization policy model. Accordingly, the third, fourth, and fifth characteristics in Table 2 are not applicable. They neither provided use case demonstration examples nor implemented their model. In general, using blockchain technology in access control is still a debatable topic; as described in [46], blockchain technology has some technical characteristics (cryptocurrency fees and processing time) that could limit its applicability. Xiaojian et al. [73] presented a high-level security protection enforcement architecture based on zero trust for power IoT. However, their architecture only governs interactions where smart devices are the access requester. Moreover, they described an access control model that is required for their proposed enforcement architecture without providing a formal authorization policy model or policy specification. Finally, they neither provided use case demonstration nor implemented their framework. Chen et al. [24] proposed a security awareness and protection system architecture that leverages zero-trust architecture specifically for a 5G-based smart medical platform. Moreover, they described an access control model that captures subject's, object's, behavioral, and environmental attributes to decide on actions. Their model supports continuous and score-based authorization. However, they didn't provide a formal policy model for their described authorization model. Their architecture and authorization model govern users-to-cloud's services interactions only. Their model is not illustrated through use case scenarios. Finally, their proposed system is implemented and tested. Singh et al. [68] introduced a modified multi-access edge computing software-defined perimeter ZT enforcement architecture for LTE networks. In this architecture, subjects are IoT devices, while resources are cloud services. Furthermore, their main focus was to propose an enforcement architecture and implement and test it. They also provided a conceptual workflow architecture that can be considered as a use case demonstration. However, they didn't propose an authorization policy model; accordingly, the third, fourth, and fifth characteristics in Table 2 are not applicable. Vanickis et al. [71] have introduced a policy enforcement framework for risk adaptive access control in ZT networks. Additionally, they specified the design of firewall policy language to express firewall rules. They designed a mechanism to map these rules to specific firewall syntax and install them on the firewall. However, unlike our approach, they did not consider different actors and resources in IoT environments where they only investigated users to

services requests. Moreover, they didn't implement or test their solution. Dimitrakos et al. [31] presented a risk or trust-aware authorization framework for IoT by extending an ABAC model. They introduced an access control enforcement architecture. They described an authorization model; however, they didn't provide a formal policy specification language for their authorization model. Their described authorization model captures different attributes and supports continuous and score-based authorization. They provided use case demonstration and experimental validation. On the other hand, they didn't consider all the different actors and resources in IoT systems. Finally, Mandal et al. [45] presented a user-to-cloud resources ZT architecture to support remote work. Furthermore, they proposed a formal access control policy model. By assigning threshold values for different port numbers, their proposed access control policy allows for inspecting the network traffic by analyzing the requesting port number against the requested port number. Their model is provided with use case demonstrations and proof of concept implementation. However, their model neither captures different attributes (actor's attributes, target's attributes, and environment attributes) nor supports continuous authorization.

8 Discussion and Conclusion

This article presents a novel ZT architecture for IoT systems (ZTA-IoT). ZTA-IoT extends and adapts NIST's ZTA framework to fully express and govern different types of interactions in cloud-enabled IoT systems. Moreover, this article proposes an access control framework denoted as ZTA-IoT-ACF. This framework captures various interactions that need to be governed between different components within and across different ZTA-IoT layers to maintain ZT principles that focus on authentication, authorization, and shrinking implicit trust zones to enhance security and privacy. Within this framework, this article then focuses on object-level interactions, i.e., interactions where the target resource is a device or an information file generated or stored by a device. Where it develops the object-level Zero Trust score-based authorization framework for IoT systems, denoted as ZTA-IoT-OL-SAF, to govern access requests in this context. ZTA-IoT-OL-SAF enables the future development of ZT access control models capable of: (i) capturing different actors, targets, actions, and context attributes, (ii) performing ongoing authorization, and (iii) dynamically deciding on access requests based on predefined authorization policy model as well as real-time calculated scores. With this framework in place, this article proposes a novel usage control authorization policy model ($UCON_{IoT}$) that controls users-to-objects and devices-to-objects interactions. $UCON_{IoT}$ is formally defined, provided with illustrative use cases, implemented, and tested using AWS IoT service. In our implementation, we adhered to the CDM2 deployment model as described in [3]. Within this deployment model, a Lambda function was employed in the cloud to execute the access control logic. Furthermore, the necessary attributes for evaluating access requests were stored and retrieved from a DynamoDB instance also located in the cloud. Future work includes investigating the virtual-object level, the cloud level, and the application level interactions. Furthermore, within ZTA-IoT-OL-SAF, a detailed study to propose score and threshold calculation algorithms should be conducted. Depending on the application system, there could be different types of score calculation and threshold calculation algorithms, and they may be probabilistic, heuristic, or simple mathematical algorithms. Additionally, an operational access control policy model that controls the system's processes module (SPM) interactions and a series of administrative access control model that governs administrators' interactions within the ZTA-IoT-OL-SAF need to be developed. Our proof of concept implementation demonstrates the effectiveness and potential of fine-grained security policies within the context of smart IoT systems. It would be beneficial to expand this study by incorporating various deployment models outlined in [3]. Additionally, deploying our model across diverse cloud service provider platforms and evaluating multiple use case scenarios is essential to assess its applicability across different environments. This approach will facilitate

the development of comprehensive usability and deployment recommendations. A critical aspect that needs to be addressed in future work is usability. Considering that many IoT systems users are usually constrained and unwilling to deal with complicated systems, a user-related study needs to be performed to define a standard preferable default setting policies. Also, a simple, usable, and expressive user interface need to be developed to mediate users' interaction with the access control system.

Appendix

A Formal Definitions

This Appendix presents the formal definitions of the $UCON_{IoT}$ policy model as described in Section 4.2. The fundamental sets and functions are formalized in Tables 3 and 4. Furthermore, Table 5 provides formal definitions of various types of constraints in $UCON_{IoT}$. The check access predicates are formally defined in Table 6, while the grammar for the attributes update function is specified in Table 7.

Table 3. $UCON_{IoT}$ Model Formalization Part I: Basic Components

Actors: Actors can be users (U) or devices (D). –U is the finite set of users. –D is the finite set of devices deployed in the smart system. –S is the finite set of sessions created by users or devices. $S = S_u \cup S_d$. –S_u is the set of users' sessions (each session is created, terminated, and controlled by an individual user). –The function $user(s_u) : S_u \rightarrow U$ maps each user session to its unique creator and controlling user. –S_d is the set of devices' sessions (each session is created, terminated, and controlled by an individual device). –The function $device(s_d) : S_d \rightarrow D$ maps each device session to its unique creator and controlling device. Targets (Objects) –OBJ is the set of objects in the system. Objects can be devices (D) or information ($INFO$), $OBJ = D \cup INFO$. –D is the set of devices defined above. –$INFO$ is the set of possible information generated by or stored on devices (device manufacturers specified). – The function $info : D \rightarrow 2^{INFO}$ specifies the valid information for each device (device manufacturers specified). Operations –OP is the set of operations in the system. We have two types of operations, operations on devices OP_d and operations on information $OP_{info}. OP = OP_d \cup OP_{info}.$ –OP_d is the set of possible operations on devices (device manufacturers specified). –The function $ops : D \rightarrow 2^{OP}$ specifies the valid operations for each device (device manufacturers specified). –OP_{info} is the set of possible operations on devices' information (device manufacturers specified). –The function $ops : INFO \rightarrow 2^{OP}$ specifies the valid operations for each information (device manufacturers specified). Permissions –P is the set of permissions in the system. We have two types of permissions, permissions on devices P_d and permissions on information P_{info}. $P = P_d \cup P_{info}$. –P_d is the set of devices' permissions. $P \subseteq OP_d \times D$, every permission is an operation, device pair (device manufacturer specified). –P_{info} is the set of information permissions. $P \subseteq OP_{info} \times INFO \times D$, every permission is an operation, information, and device triplets (device manufacturer specified). Context States –$CS = \{current\}$ is a singleton set where <i>current</i> denotes the context at the current time instance. Attribute Functions and Values –$UA, USA, DA, DSA, OPA, INFOA$ and CSA are sets of user, user's session, device, device's session (when the device is an actor), operation, information, and context state attribute functions respectively. –User's session attribute functions can be inherited attribute functions from the session's unique user creator or it can be unique users' session attribute functions $USA = IUA \cup S_uA$. –IUA is the inherited user attribute functions set, where: $IUA \subseteq UA$. –Each session s_u inherits a subset of the attribute functions in UA from its unique user creator (controlled by the session creator $user(s_u)$). For every inherited attribute function $att \in IUA$, $att(s_u) = att(user(s_u))$ at all times. Unless otherwise specified use of a non-inherited user attribute in a logical formula renders that formula false. –S_uA is the unique users' session attribute functions set. –Device's session attribute functions can be inherited attribute functions from the session's unique device creator or it can be unique devices' session attribute functions $DSA = IDA \cup S_dA$. –IDA is the Inherited device attribute functions set, where: $IDA \subseteq DA$. –Each session s_d inherits a subset of the attribute functions in DA from its unique device creator (controlled by the session creator $device(s_d)$). For every inherited attribute function $att \in IDA$, $att(s_d) = att(device(s_d))$ at all times. Unless otherwise specified use of a non-inherited device attribute in a logical formula renders that formula false. –S_dA is the unique devices' session attribute functions set. –For each attribute att in $UA \cup USA \cup DA \cup DSA \cup OPA \cup INFOA \cup CSA$, $Range(att)$ is the attribute range, a finite set of atomic values. –$attType : UA \cup USA \cup DA \cup DSA \cup OPA \cup INFOA \cup CSA \rightarrow \{set, atomic\}$. –Each $att \in UA \cup USA \cup DA \cup DSA \cup OPA \cup INFOA \cup CSA$ correspondingly maps users in U, sessions in S_u, devices in D, sessions in S_d, operations in OP, information in $INFO$, or the context state <i>current</i> to atomic or set attribute values. Formally: $att : U \text{ or } S_u \text{ or } D \text{ or } S_d \text{ or } OP \text{ or } INFO \text{ or } \{current\} \rightarrow \begin{cases} Range(att), & \text{if } attType(att) = atomic \\ 2^{Range(att)}, & \text{if } attType(att) = set \end{cases}$ –Every $att \in UA \cup USA \cup DA \cup DSA \cup OPA \cup INFOA \cup CSA$, att is designated to be either a static or a dynamic attribute. –Static attribute ranges and values are set and changed by administrator actions. –Dynamic attribute ranges and values automatically determined by sensors deployed in the smart system.
--

Table 4. $UCON_{IoT}$ Model Formalization Part II: Basic Components**Guardian Attribute Functions**

–We have two types of guardian attribute functions: user guardian attribute functions UGA and device guardian attribute functions DGA .

– UGA and DGA are sets of user and device guardian attribute functions, respectively.

– For each guardian attribute function ga in $UGA \cup DGA$, $Range(ga)$ is the guardian attribute range, a finite set of atomic values.

– $gaType : UGA \cup DGA \rightarrow \{set, atomic\}$.

–Each $ga \in UGA \cup DGA$ correspondingly maps users in U and devices in D to atomic or set guardian attribute values. Formally:

$$ga : U \text{ or } D \rightarrow \begin{cases} Range(ga), & \text{if } gaType(ga) = atomic \\ 2^{Range(ga)}, & \text{if } gaType(ga) = set \end{cases}$$

U-D Attributes Authorization Function

– $Authorization_{U-D}(s_u : S_u, op : OP, d : D, current : CS)$ is a logic formula defined using the grammar defined below (system administrator

specified). It is evaluated for a specific session s_u , operation op_k , device d_j , and context state $current$ as specified in the following grammar.

D-D Attributes Authorization Function

– $Authorization_{D-D}(s_d : S_d, op : OP, d : D, current : CS)$ is a logic formula defined using the grammar defined below (System administrator

specified). It is evaluated for a specific session s_d , operation op_k , device d_j , and context state $current$ as specified in the following grammar.

U-INFO Attributes Authorization Function

– $Authorization_{U-INFO}(s_u : S_u, op : OP, d : D, info : INFO, current : CS)$ is a logic formula defined using the grammar defined below (system administrator specified). It is evaluated for a specific session s_u , operation op_k , device d_j , information $info_i$, and context state $current$ as specified in the following grammar.

D-INFO Attributes Authorization Function

– $Authorization_{D-INFO}(s_d : S_d, op : OP, d : D, info : INFO, current : CS)$ is a logic formula defined using the grammar defined below (System administrator specified). It is evaluated for a specific session s_d , operation op_k , device d_j , information $info_i$, and context state $current$ as specified in the following grammar.

– Attribute authorization functions whether it is U-D, D-D, U-INFO, or D-INFO are propositional logic formulas specified using the following grammar.

- $\alpha ::= term \mid term \wedge term \mid term \vee term \mid (term) \mid \neg term \mid \exists x \in set. \alpha \mid \forall x \in set. \alpha$
- $term ::= set \mid setcompare \mid set \mid atomic \in set \mid atomic \notin set \mid atomic \mid atomiccompare \mid atomic$
- $setcompare ::= < \mid \leq \mid \geq \mid >$
- $atomiccompare ::= < \mid \leq \mid \geq \mid >$
- The definition of the variable set have four case:
 - In the case of U-D access: $set ::= usa(s_u) \mid opa(op) \mid da(d) \mid csa(current)$
 - In the case of D-D access: $set ::= dsa(s_d) \mid opa(op) \mid da(d) \mid csa(current)$
 - In the case of U-INFO access: $set ::= usa(s_u) \mid opa(op) \mid da(d) \mid infoa(info) \mid csa(current)$
 - In the case of D-INFO access: $set ::= dsa(s_d) \mid opa(op) \mid da(d) \mid infoa(info) \mid csa(current)$
- Where $attType(usa) = attType(dsa) = attType(opa) = attType(csa) = attType(da) = attType(infoa) = attType(pa) = set$
- The definition of the variable atomic have four case:
 - In the case of U-D access: $atomic ::= usa(s_u) \mid opa(op) \mid da(d) \mid csa(current) \mid value$
 - In the case of D-D access: $atomic ::= dsa(s_d) \mid opa(op) \mid da(d) \mid csa(current) \mid value$
 - In the case of U-INFO access: $atomic ::= usa(s_u) \mid opa(op) \mid da(d) \mid infoa(info) \mid csa(current) \mid value$
 - In the case of D-INFO access: $atomic ::= dsa(s_d) \mid opa(op) \mid da(d) \mid infoa(info) \mid csa(current) \mid value$
- Where $attType(usa) = attType(dsa) = attType(opa) = attType(csa) = attType(da) = attType(infoa) = atomic$

–For a specific session s_u , device d_j , and operation op_k the authorization function $Authorization_{U-D}(s_u, op_k, d_j, current)$ is evaluated by substituting the actual attribute values of $usa(s_u)$, $opa(op_k)$, $da(d_j)$, and $csa(current)$ for the corresponding symbolic placeholders and evaluating the resulting logical formula to be True or False. Any term that references an undefined attribute value is evaluated as False.

–For a specific session s_d , device d_j , and operation op_k the authorization function $Authorization_{D-D}(s_d, op_k, d_j, current)$ is evaluated by substituting the actual attribute values of $dsa(s_d)$, $opa(op_k)$, $da(d_j)$, and $csa(current)$ for the corresponding symbolic placeholders and evaluating the resulting logical formula to be True or False. Any term that references an undefined attribute value is evaluated as False.

–For a specific session s_u , device d_j , information $info_i$, and operation op_k the authorization function $Authorization_{U-INFO}(s_u, op_k, d_j, info_i, current)$ is evaluated by substituting the actual attribute values of $usa(s_u)$, $opa(op_k)$, $da(d_j)$, $infoa(info_i)$, and $csa(current)$ for the corresponding symbolic placeholders and evaluating the resulting logical formula to be True or False. Any term that references an undefined attribute value is evaluated as False.

–For a specific session s_d , device d_j , information $info_i$ and operation op_k the authorization function $Authorization_{D-INFO}(s_d, op_k, d_j, info_i, current)$ is evaluated by substituting the actual attribute values of $dsa(s_d)$, $opa(op_k)$, $da(d_j)$, $infoa(info_i)$, and $csa(current)$ for the corresponding symbolic placeholders and evaluating the resulting logical formula to be True or False. Any term that references an undefined attribute value is evaluated as False.

Table 5. $UCON_{IoT}$ Model Formalization Part III: Constraints

Constraints: A constraint is an invariant that must be maintained at all times.

Users Attributes Constraints:

– $UAC_{Constraint} \subseteq UAP \times 2^{UAP}$ is the user attribute constraints relation (system administrator specified), where

$$UAP = \{(ua, v) \mid ua \in UA \wedge v \in Range(ua)\}$$

Each $uac = ((ua_x, v_y), UAP_j) \in UAC_{Constraint}$ specifies the following invariant:

$$\begin{cases} (\forall u_l \in U)(\forall (ua_m, v_n) \in UAP_j)[ua_x(u_l) = v_y \Rightarrow ua_m(u_l) \neq v_n], & \text{if } attType(ua_x) = attType(ua_m) = atomic \\ (\forall u_l \in U)(\forall (ua_m, v_n) \in UAP_j)[ua_x(u_l) = v_y \Rightarrow v_n \notin ua_m(u_l)], & \text{if } attType(ua_x) = atomic, attType(ua_m) = set \\ (\forall u_l \in U)(\forall (ua_m, v_n) \in UAP_j)[v_y \in ua_x(u_l) \Rightarrow v_n \notin ua_m(u_l)], & \text{if } attType(ua_x) = attType(ua_m) = set \\ (\forall u_l \in U)(\forall (ua_m, v_n) \in UAP_j)[v_y \in ua_x(u_l) \Rightarrow ua_m(u_l) \neq v_n], & \text{if } attType(ua_x) = set, attType(ua_m) = atomic \end{cases}$$

Devices Attributes Constraints

– $DAC_{Constraint} \subseteq DAP \times 2^{DAP}$ is the device attribute constraints relation (system administrator specified), where

$$DAP = \{(da, v) \mid da \in DA \wedge v \in Range(da)\}$$

Each $dac = ((da_x, v_y), DAP_j) \in DAC_{Constraint}$ specifies the following invariant:

$$\begin{cases} (\forall d_l \in D)(\forall (dam, v_n) \in DAP_j)[da_x(d_l) = v_y \Rightarrow dam(d_l) \neq v_n], & \text{if } attType(da_x) = attType(dam) = atomic \\ (\forall d_l \in D)(\forall (dam, v_n) \in DAP_j)[da_x(d_l) = v_y \Rightarrow v_n \notin dam(d_l)], & \text{if } attType(da_x) = atomic, attType(dam) = set \\ (\forall d_l \in D)(\forall (dam, v_n) \in DAP_j)[v_y \in da_x(d_l) \Rightarrow v_n \notin dam(d_l)], & \text{if } attType(da_x) = attType(dam) = set \\ (\forall d_l \in D)(\forall (dam, v_n) \in DAP_j)[v_y \in da_x(d_l) \Rightarrow dam(d_l) \neq v_n], & \text{if } attType(da_x) = set, attType(dam) = atomic \end{cases}$$

User's Sessions Attributes Constraints

– $USAP_{Constraint} \subseteq USAP \times 2^{USAP}$ is the users' session attribute constraints relation (system administrator specified), where

$$USAP = \{(usa, v) \mid usa \in USA \wedge v \in Range(usa)\}$$

Each $usac = ((usa_x, v_y), USAP_j) \in USAP_{Constraint}$ specifies the following invariant:

$$\begin{cases} (\forall s_l \in S_u)(\forall (usam, v_n) \in USAP_j)[usa_x(s_l) = v_y \Rightarrow usam(s_l) \neq v_n], & \text{if } attType(usa_x) = attType(usam) = atomic \\ (\forall s_l \in S_u)(\forall (usam, v_n) \in USAP_j)[usa_x(s_l) = v_y \Rightarrow v_n \notin usam(s_l)], & \text{if } attType(usa_x) = atomic, attType(usam) = set \\ (\forall s_l \in S_u)(\forall (usam, v_n) \in USAP_j)[v_y \in usa_x(s_l) \Rightarrow v_n \notin usam(s_l)], & \text{if } attType(usa_x) = attType(usam) = set \\ (\forall s_l \in S_u)(\forall (usam, v_n) \in USAP_j)[v_y \in usa_x(s_l) \Rightarrow usam(s_l) \neq v_n], & \text{if } attType(usa_x) = set, attType(usam) = atmic \end{cases}$$

Device's Sessions Attributes Constraints

– $DSAP_{Constraint} \subseteq DSAP \times 2^{DSAP}$ is the devices' session attribute constraints relation (system administrator specified), where:

$$DSAP = \{(dsa, v) \mid dsa \in DSA \wedge v \in Range(dsa)\}$$

Each $dsac = ((dsa_x, v_y), DSAP_j) \in DSAP_{Constraint}$ specifies the following invariant:

$$\begin{cases} (\forall s_l \in S_d)(\forall (dsam, v_n) \in DSAP_j)[dsa_x(s_l) = v_y \Rightarrow dsam(s_l) \neq v_n], & \text{if } attType(dsa_x) = attType(dsam) = atomic \\ (\forall s_l \in S_d)(\forall (dsam, v_n) \in DSAP_j)[dsa_x(s_l) = v_y \Rightarrow v_n \notin dsam(s_l)], & \text{if } attType(usa_x) = atomic, attType(usam) = set \\ (\forall s_l \in S_d)(\forall (dsam, v_n) \in DSAP_j)[v_y \in dsa_x(s_l) \Rightarrow v_n \notin dsam(s_l)], & \text{if } attType(dsa_x) = attType(dsam) = set \\ (\forall s_l \in S_d)(\forall (dsam, v_n) \in DSAP_j)[v_y \in dsa_x(s_l) \Rightarrow dsam(s_l) \neq v_n], & \text{if } attType(dsa_x) = set, attType(dsam) = atmic \end{cases}$$

Users' Permissions Constraints

– User guardian attribute pairs are set of user guardian attribute function and value pairs, where:

$$UGAP = \{(uga, v) \mid uga \in UGA \wedge v \in Range(uga)\}$$

– Users' Permissions Constraints are defined as many to many subset of permissions to set of user guardian attribute pairs:

$$UPC_{Constraint} \subseteq 2^P \times 2^{UGAP}$$

Devices' Permissions Constraints

– Device guardian attribute pairs are set of device guardian attribute function and value pairs, where:

$$DGAP = \{(dga, v) \mid dga \in DGA \wedge v \in Range(dga)\}$$

– Devices' Permissions Constraints are defined as many to many subset of permissions to set of device guardian attribute pairs:

$$DPC_{Constraint} \subseteq 2^P \times 2^{DGAP}$$

Permission Constraints Checking Function

– Depending on the four combinations of actors and targets, we have four check permission constraints functions:

– $CheckPC_{Constraints}(s_{u_i}, op_k, d_j)$ evaluates to True or False using the following formulas:

$$\begin{cases} (\forall (P_x, UGAP_y) \in UPC_{Constraint})(\forall (uga_m, v_n) \in UGAP_y)[((op_k, d_j) \notin P_x) \vee (uga_m(user(s_{u_i})) \neq v_n)], & \text{if } attType(uga_m) = atomic \\ (\forall (P_x, UGAP_y) \in UPC_{Constraint})(\forall (uga_m, v_n) \in UGAP_y)[((op_k, d_j) \notin P_x) \vee (v_n \notin uga_m(user(s_{u_i}))), & \text{if } attType(uga_m) = set \end{cases}$$

– $CheckPC_{Constraints}(s_{u_i}, op_k, d_j, info_i)$ evaluates to True or False using the following formulas:

$$\begin{cases} (\forall (P_x, UGAP_y) \in UPC_{Constraint})(\forall (uga_m, v_n) \in UGAP_y)[((info_i, op_k, d_j) \notin P_x) \vee (uga_m(user(s_{u_i})) \neq v_n)], & \text{if } attType(uga_m) = atomic \\ (\forall (P_x, UGAP_y) \in UPC_{Constraint})(\forall (uga_m, v_n) \in UGAP_y)[((info_i, op_k, d_j) \notin P_x) \vee (v_n \notin uga_m(user(s_{u_i}))), & \text{if } attType(uga_m) = set \end{cases}$$

– $CheckPC_{Constraints}(s_{d_i}, op_k, d_j)$ evaluates to True or False using the following formulas:

$$\begin{cases} (\forall (P_x, DGAP_y) \in DPC_{Constraint})(\forall (dga_m, v_n) \in DGAP_y)[((op_k, d_j) \notin P_x) \vee (dga_m(device(s_{d_i})) \neq v_n)], & \text{if } attType(dga_m) = atomic \\ (\forall (P_x, DGAP_y) \in DPC_{Constraint})(\forall (dga_m, v_n) \in DGAP_y)[((op_k, d_j) \notin P_x) \vee (v_n \notin dga_m(device(s_{d_i}))), & \text{if } attType(dga_m) = set \end{cases}$$

– $CheckPC_{Constraints}(s_{d_i}, op_k, d_j, info_i)$ evaluates to True or False using the following formulas:

$$\begin{cases} (\forall (P_x, DGAP_y) \in DPC_{Constraint})(\forall (dga_m, v_n) \in DGAP_y)[((op_k, info_i, d_j) \notin P_x) \vee (dga_m(device(s_{d_i})) \neq v_n)], & \text{if } attType(dga_m) = atomic \\ (\forall (P_x, DGAP_y) \in DPC_{Constraint})(\forall (dga_m, v_n) \in DGAP_y)[((op_k, info_i, d_j) \notin P_x) \vee (v_n \notin dga_m(device(s_{d_i}))), & \text{if } attType(dga_m) = set \end{cases}$$

Table 6. $UCON_{IoT}$ Model Formalization Part IV: Check Access Predicates**Check Access Predicate**

– Depending on the four combinations of actors and targets, we have four check access predicates:

– $CheckAccess(s_{u_i}, op_k, d_j, current)$ is evaluated when a session s_{u_i} attempts an operation op_k on a device d_j in conditions of context state $current$. It evaluates to True or False using the following formula:

$$op_k \in ops(d_j) \wedge Authorization_{U-D}(s_{u_i}, op_k, d_j, current) \wedge CheckPConstraints(s_{u_i}, op_k, d_j)$$

– $CheckAccess(s_{u_i}, op_k, d_j, info_I, current)$ is evaluated when a session s_{u_i} attempts an operation op_k on information $info_I$ on a device d_j in conditions of context state $current$. It evaluates to True or False using the following formula:

$$op_k \in ops(info_I) \wedge info_I \in info(d_j) \wedge Authorization_{U-INFO}(s_{u_i}, op_k, d_j, info_I, current) \wedge CheckPConstraints(s_{u_i}, op_k, d_j, info_I)$$

– $CheckAccess(s_{d_i}, op_k, d_j, current)$ is evaluated when a session s_{d_i} attempts an operation op_k on a device d_j in conditions of context state $current$. It evaluates to True or False using the following formula:

$$op_k \in ops(d_j) \wedge Authorization_{D-D}(s_{d_i}, op_k, d_j, current) \wedge CheckPConstraints(s_{d_i}, op_k, d_j)$$

– $CheckAccess(s_{d_i}, op_k, d_j, info_I, current)$ is evaluated when a session s_{d_i} attempts an operation op_k on information $info_I$ on a device d_j in conditions of context state $current$. It evaluates to True or False using the following formula:

$$op_k \in ops(info_I) \wedge info_I \in info(d_j) \wedge Authorization_{D-INFO}(s_{d_i}, op_k, d_j, info_I, current) \wedge CheckPConstraints(s_{d_i}, op_k, d_j, info_I)$$

Table 7. $UCON_{IoT}$ Model Formalization Part V: Attributes Update Functions Grammar

Attributes Update Functions Pre execution update functions (PreUpdate): We have four preupdate functions: –$PreUpdate_{U-D_{op}}(s : S_u, d : D, current : CS)$ is a function evaluated for a specific session $s \in S_u$, operation op, device d and context state $current$ if the authorization function is satisfied and before access is enforced. It is defined using the conditional statement γ defined below. –$PreUpdate_{D-D_{op}}(s : S_d, d : D, current : CS)$ is a function evaluated for a specific session $s \in S_d$, operation op, device d and context state $current$ if the authorization function is satisfied and before access is enforced. It is defined using the conditional statement γ defined below. –$PreUpdate_{U-INFO_{op}}(s : S_u, d : D, info : INFO, current : CS)$ is a function evaluated for a specific session $s \in S_u$, operation op, device d, information $info$, and context state $current$ if the authorization function is satisfied and before access is enforced. It is defined using the conditional statement γ defined below. –$PreUpdate_{D-INFO_{op}}(s : S_d, d : D, info : INFO, current : CS)$ is a function evaluated for a specific session $s \in S_d$, operation op, device d, information $info$, and context state $current$ if the authorization function is satisfied and before access is enforced. It is defined using the conditional statement γ defined below. During execution update functions (OnUpdate): We have four OnUpdate functions: –$OnUpdate_{U-D_{op}}(s : S_u, d : D, current : CS)$ is a function executed while a session $s \in S_u$ executes the specified operation op on the device d during the context state $current$. It is defined using the conditional statement γ defined below. –$OnUpdate_{D-D_{op}}(s : S_d, d : D, current : CS)$ is a function executed while a session $s \in S_d$ executes the specified operation op on the device d during the context state $current$. It is defined using the conditional statement γ defined below. –$OnUpdate_{U-INFO_{op}}(s : S_u, d : D, info : INFO, current : CS)$ is a function executed while a session $s \in S_u$ executes the specified operation op on the information $info$ which stored in the device d during the context state $current$. It is defined using the conditional statement γ defined below. –$OnUpdate_{D-INFO_{op}}(s : S_d, d : D, info : INFO, current : CS)$ is a function executed while a session $s \in S_d$ executes the specified operation op on the information $info$ which stored in the device d during the context state $current$. It is defined using the conditional statement γ defined below. Post execution update functions (PostUpdate): We have four types of PostUpdate functions: –$PostUpdate_{U-D_{op}}(s : S_u, d : D, current : CS)$ is a function executed after a session $s \in S_u$ executes the specified operation op on the device d during the context state $current$. It is defined using the conditional statement γ defined below. –$PostUpdate_{D-D_{op}}(s : S_d, d : D, current : CS)$ is a function executed after a session $s \in S_d$ executes the specified operation op on the device d during the context state $current$. It is defined using the conditional statement γ defined below. –$PostUpdate_{U-INFO_{op}}(s : S_u, d : D, info : INFO, current : CS)$ is a function executed after a session $s \in S_u$ executes the specified operation op on the information $info$ which stored in the device d during the context state $current$. It is defined using the conditional statement γ defined below. –$PostUpdate_{D-INFO_{op}}(s : S_d, d : D, info : INFO, current : CS)$ is a function executed after a session $s \in S_d$ executes the specified operation op on the information $info$ which stored in the device d during the context state $current$. It is defined using the conditional statement γ defined below. Attributes Update Functions' Grammar $\gamma ::= \beta \rightarrow \delta$ $\beta ::= term \mid term \wedge term \mid term \vee term \mid (term) \mid \neg term \mid \exists x \in set. \beta \mid \forall x \in set. \beta$ $term ::= set \ setcompare \ set \mid atomic \in set \mid atomic \notin set \mid atomic \ atomiccompare \ atomic$ $setcompare ::= c \mid \leq \mid \geq$ $atomiccompare ::= < \mid \leq \mid \geq \mid >$ $\delta ::= \delta \wedge \delta \mid \delta$ $\delta ::= atomic \ assignmentoperation \ atomic \ atomicoperation \ atomic \mid set \ assignmentoperation \ set \ setoperation \ set$ $assignmentoperation ::= =$ $setOperation ::= \cup \mid \cap \mid \setminus$ $atomicOperation ::= + \mid - \mid \times \mid \div \mid \bmod$ - The definition of the variable set have four cases: - In the case of U-D access: $set ::= usa(s_u) \mid opa(op) \mid da(d) \mid csa(current)$ - In the case of D-D access: $set ::= dsa(s_d) \mid opa(op) \mid da(d) \mid csa(current)$ - In the case of U-INFO access: $set ::= usa(s_u) \mid opa(op) \mid da(d) \mid infoa(info) \mid csa(current)$ - In the case of D-INFO access: $set ::= dsa(s_d) \mid opa(op) \mid da(d) \mid infoa(info) \mid csa(current)$ Where $attType(usa) = attType(dsa) = attType(opa) = attType(csa) = attType(da) = attType(infoa) = attType(pa) = set$ - The definition of the variable atomic have four cases: - In the case of U-D access: $atomic ::= usa(s_u) \mid opa(op) \mid da(d) \mid csa(current) \mid value$ - In the case of D-D access: $atomic ::= dsa(s_d) \mid opa(op) \mid da(d) \mid csa(current) \mid value$ - In the case of U-INFO access: $atomic ::= usa(s_u) \mid opa(op) \mid da(d) \mid infoa(info) \mid csa(current) \mid value$ - In the case of D-INFO access: $atomic ::= dsa(s_d) \mid opa(op) \mid da(d) \mid infoa(info) \mid csa(current) \mid value$ Where $attType(usa) = attType(dsa) = attType(opa) = attType(csa) = attType(da) = attType(infoa) = atomic$

References

- [1] 2023. AWS-IoT. <https://aws.amazon.com/iot/>
- [2] Tahir Ahmad, Umberto Morelli, Silvio Ranise, and Nicola Zannone. 2018. A lazy approach to access control as a service (ACaaS) for IoT: An AWS case study. In *23rd ACM on Symposium on Access Control Models and Technologies*. 235–246.
- [3] Tahir Ahmad, Umberto Morelli, Silvio Ranise, and Nicola Zannone. 2022. Extending access control in AWS IoT through event-driven functions: An experimental evaluation using a smart lock system. *International Journal of Information Security* 21, 2 (2022), 379–408.
- [4] Ala Al-Fuqaha, Mohsen Guizani, Mehdi Mohammadi, Mohammed Aledhari, and Moussa Ayyash. 2015. Internet of Things: A survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials* 17, 4 (2015), 2347–2376.
- [5] Fadele Ayotunde Alaba, Mazliza Othman, Ibrahim Abaker Targio Hashem, and Faiz Alotaibi. 2017. Internet of Things security: A survey. *Journal of Network and Computer Applications* 88 (2017), 10–28.

- [6] Asma Alshehri and Ravi Sandhu. 2016. Access control models for cloud-enabled Internet of Things: A proposed architecture and research agenda. In *2016 IEEE 2nd International Conference on Collaboration and Internet Computing (CIC'16)*. IEEE, 530–538.
- [7] Safwa Ammer, James Benson, and Ravi Sandhu. 2022. Hybrid approaches (ABAC and RBAC) toward secure access control in smart home IoT. *IEEE Transactions on Dependable and Secure Computing* (2022).
- [8] Safwa Ameer, James Benson, and Ravi Sandhu. 2020. The EGRBAC model for smart home IoT. In *2020 IEEE 21st International Conference on Information Reuse and Integration for Data Science (IRI)*. IEEE.
- [9] Safwa Ameer, James Benson, and Ravi Sandhu. 2022. An attribute-based approach toward a secured smart-home IoT access control and a comparison with a role-based approach. *Information* 13, 2 (2022), 60.
- [10] Safwa Ameer, Maanak Gupta, Smriti Bhatt, and Ravi Sandhu. 2022. BlueSky: Towards convergence of zero trust principles and score-based authorization for IoT enabled smart systems. In *Proceedings of the 27th ACM on Symposium on Access Control Models and Technologies*. 235–244.
- [11] Safwa Ameer, Ram Krishnan, Ravi Sandhu, and Maanak Gupta. 2023. Utilizing the DLBAC approach toward a ZT score-based authorization for IoT systems. In *13th ACM Conference on Data and Application Security and Privacy*. 283–285.
- [12] Safwa Ameer and Ravi Sandhu. 2021. The HABAC model for smart home IoT and comparison to EGRBAC. In *ACM Workshop on Secure and Trustworthy Cyber-Physical Systems (SAT-CPS'21)*.
- [13] Orlando Arias, Jacob Wurm, Khoa Hoang, and Yier Jin. 2015. Privacy and security in internet of things and wearable devices. *TMSCS* (2015).
- [14] Luigi Atzori, Antonio Iera, and Giacomo Morabito. 2010. The Internet of Things: A survey. *Computer Networks* 54, 15 (2010), 2787–2805.
- [15] Syafril Bandara, Takeshi Yashiro, Noboru Koshizuka, and Ken Sakamura. 2016. Access control framework for api-enabled devices in smart buildings. In *APCC*. IEEE.
- [16] Ezedine Barka, Sujith Samuel Mathew, and Yacine Atif. 2015. Securing the web of things with role-based access control. In *C2SI*. Springer.
- [17] Bruhadeshwar Bezawada, Kyle Haefner, and Indrakshi Ray. 2018. Securing home IoT environments with attribute-based access control. In *ABAC'18*. ACM.
- [18] Smriti Bhatt, Farhan Patwa, and Ravi Sandhu. 2017. An access control framework for cloud-enabled wearable internet of things. In *2017 IEEE 3rd International Conference on Collaboration and Internet Computing (CIC'17)*. IEEE, 328–338.
- [19] Smriti Bhatt, Farhan Patwa, and Ravi Sandhu. 2017. Access control model for AWS Internet of Things. In *International Conference on Network and System Security*.
- [20] Smriti Bhatt and Ravi Sandhu. 2020. ABAC-CC: Attribute-based access control and communication control for Internet of Things. In *Proceedings of the 25th ACM Symposium on Access Control Models and Technologies*.
- [21] Smriti Bhatt and Ravi Sandhu. 2020. Convergent access control to enable secure smart communities. In *TPS-ISA*. IEEE.
- [22] Alessio Botta, Walter De Donato, Valerio Persico, and Antonio Pescapé. 2016. Integration of cloud computing and Internet of Things: A survey. *Future Generation Computer Systems* 56 (2016), 684–700.
- [23] Z. Berkay Celik, Gang Tan, and Patrick D. McDaniel. 2019. IoTGuard: Dynamic enforcement of security and safety policy in commodity IoT. In *NDSS*.
- [24] Baozhan Chen, Siyuan Qiao, Jie Zhao, Dongqing Liu, Xiaobing Shi, Minzhao Lyu, Haotian Chen, Huimin Lu, and Yunkai Zhai. 2020. A security awareness and protection system for 5G smart healthcare based on zero-trust architecture. *IEEE Internet of Things Journal* 8, 13 (2020), 10248–10263.
- [25] Zhiyu Chen, Longchuan Yan, Zitong Lü, Yanling Zhang, Yonghe Guo, Wenjing Liu, and Jiaxing Xuan. 2021. Research on zero-trust security protection technology of power IoT based on blockchain. In *Journal of Physics: Conference Series*, Vol. 1769. IOP Publishing, 012039.
- [26] Pietro Colombo, Elena Ferrari, and Engin Deniz Tümer. 2021. Access control enforcement in IoT: State of the art and open challenges in the Zero Trust era. In *2021 3rd IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA'21)*. IEEE, 159–166.
- [27] Michael J. Covington, Matthew James Moyer, and Mustaque Ahamad. 2000. *Generalized Role-based Access Control for Securing Future Applications*. Technical Report. Georgia Tech.
- [28] Dorothy E. Denning. 1976. A lattice model of secure information flow. *Commun. ACM* 19, 5 (1976), 236–243.
- [29] Tamara Denning, Tadayoshi Kohno, and Henry M. Levy. 2013. Computer security and the modern home. *Commun. ACM* (2013).
- [30] Suparna Dhar and Indranil Bose. 2021. Securing IoT devices using zero trust and blockchain. *Journal of Organizational Computing and Electronic Commerce* 31, 1 (2021), 18–34.
- [31] Theo Dimitrakos, Tezcan Dilshener, Alexander Kravtsov, Antonio La Marra, Fabio Martinelli, Athanasios Rizos, Alessandro Rosett, and Andrea Saracino. 2020. Trust aware continuous authorization for zero trust in consumer Internet of Things. In *TrustCom*. IEEE.

- [32] Earlence Fernandes, Jaeyeon Jung, and Atul Prakash. 2016. Security analysis of emerging smart home applications. In *SP*. IEEE.
- [33] Earlence Fernandes, Justin Paupore, Amir Rahmati, Daniel Simionato, Mauro Conti, and Atul Prakash. 2016. Flowfence: Practical data protection for emerging IoT application frameworks. In *25th USENIX Security Symposium*.
- [34] Maribel Fernández, Alex Franch Tapia, Jenjira Jaimunk, Manuel Martinez Chamorro, and Bhavani Thuraisingham. 2020. A data access model for privacy-preserving cloud-IoT architectures. In *SACMAT'20*.
- [35] David F. Ferraiolo, Ravi Sandhu, Serban Gavrila, D. Richard Kuhn, and Ramaswamy Chandramouli. 2001. Proposed NIST standard for role-based access control. *TISSEC* (2001).
- [36] Jorge Granjal, Edmundo Monteiro, and Jorge S. Silva. 2015. Security for the Internet of Things: A survey of existing protocols and open research issues. *IEEE Comm. Surv. Tuts* (2015).
- [37] Maanak Gupta and Ravi Sandhu. 2018. Authorization framework for secure cloud assisted connected cars and vehicular internet of things. In *ACM SACMAT*.
- [38] Maanak Gupta and Ravi Sandhu. 2021. Towards activity-centric access control for smart collaborative ecosystems. In *26th ACM Symposium on Access Control Models and Technologies*. 155–164.
- [39] Grant Ho, Derek Leung, Pratyush Mishra, Ashkan Hosseini, Dawn Song, and David Wagner. 2016. Smart locks: Lessons for securing commodity internet of things devices. In *ASIA CCS'16*. ACM.
- [40] Vincent C. Hu, D. Richard Kuhn, David F. Ferraiolo, and Jeffrey Voas. 2015. Attribute-based access control. *Comp.* (2015).
- [41] X. Jin, R. Krishnan, and R. Sandhu. 2012. A unified attribute-based access control model covering DAC, MAC and RBAC. In *IFIP Annual Conf. on Data and App. Sec.*
- [42] Qi Jing, Athanasios V. Vasilakos, Jiafu Wan, Jingwei Lu, and Dechao Qiu. 2014. Security of the Internet of Things: Perspectives and challenges. *Wireless Networks* 20, 8 (2014), 2481–2501.
- [43] D. Richard Kuhn, Edward J. Coyne, and Timothy R. Weil. 2010. Adding attributes to role-based access control. *Computer* (2010).
- [44] R. Levin, E. Cohen, W. Corwin, F. Pollack, and W. Wulf. 1975. Policy/Mechanism separation in Hydra. *SIGOPS Oper. Syst. Rev.* 9, 5 (Nov. 1975), 132–140. <https://doi.org/10.1145/1067629.806531>
- [45] Sudakshina Mandal, Danish Ali Khan, and Sarika Jain. 2021. Cloud-based zero trust access control policy: An approach to support work-from-home driven by COVID-19 pandemic. *New Generation Computing* 39, 3 (2021), 599–622.
- [46] Oscar Novo. 2018. Blockchain meets IoT: An architecture for scalable access management in IoT. *IEEE IoT Journal* (2018).
- [47] Aafaf Ouaddah, Hajar Mousannif, Anas Abou Elkalam, and Abdellah Ait Ouahman. 2017. Access control in the Internet of Things: Big challenges and new opportunities. *Comp. NW* 112 (2017).
- [48] Jaehong Park and Ravi Sandhu. 2004. The UCONABC usage control model. *ACM Trans. Inf. Syst. Secur.* 7, 1 (Feb. 2004), 128–174. <https://doi.org/10.1145/984334.984339>
- [49] Jaehong Park, Ravi Sandhu, and Yuan Cheng. 2011. ACON: Activity-centric access control for social computing. In *ARES*. IEEE.
- [50] Keyur K. Patel and Sunil M. Patel. 2016. Internet of things-IOT: Definition, characteristics, architecture, enabling technologies, application & future challenges. *International Journal of Engineering Science and Computing* 6, 5 (2016).
- [51] Pawani Porambage, Mika Ylianttila, Corinna Schmitt, Pardeep Kumar, Andrei Gurtov, and Athanasios V. Vasilakos. 2016. The quest for privacy in the Internet of Things. *IEEE Cloud Computing* (2016).
- [52] Jing Qiu, Zhihong Tian, Chunlai Du, Qi Zuo, Shen Su, and Binxiang Fang. 2020. A survey on access control in the age of Internet of Things. *IEEE Internet of Things Journal* (2020).
- [53] B. B. Prahlada Rao, Paval Saluia, Neetu Sharma, Ankit Mittal, and Shivay Veer Sharma. 2012. Cloud computing for Internet of Things & sensing based applications. In *2012 6th International Conference on Sensing Technology (ICST'12)*. IEEE, 374–380.
- [54] Sowmya Ravidas, Alexios Lekidis, Federica Paci, and Nicola Zannone. 2019. Access control in Internet-of-Things: A survey. *Journal of Network and Computer Applications* 144 (2019), 79–101.
- [55] Rodrigo Roman, Jianying Zhou, and Javier Lopez. 2013. On the features and challenges of security and privacy in distributed Internet of Things. *Computer Networks* (2013).
- [56] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. 2020. *Zero Trust Architecture*. Technical Report NIST Special Publication (SP) 800-207. National Institute of Standards and Technology.
- [57] Mayra Samaniego and Ralph Deters. 2018. Zero-trust hierarchical management in IoT. In *2018 IEEE International Congress on Internet of Things (ICIOT'18)*. IEEE, 88–95.
- [58] Ravi Sandhu. 1996. Rationale for the RBAC96 family of access control models. In *1st ACM Workshop on Role-based Access Control*. 9 pages.
- [59] R. Sandhu. 1998. Role-based access control. In *Advances in Computers*. Vol. 46.

- [60] Ravi Sandhu. 2000. Engineering authority and trust in cyberspace: The OM-AM and RBAC way. In *5th ACM Workshop on Role-based Access Control*. 111–119.
- [61] Ravi Sandhu. 2009. The PEI framework for application-centric security. In *2009 5th International Conference on Collaborative Computing: Networking, Applications and Worksharing*. IEEE, 1–5.
- [62] Ravi Sandhu and Venkata Bhamidipati. 2008. The ASCAA principles for next-generation role-based access control. In *3rd International Conference on Availability, Reliability and Security (ARES'08)*. 6 pages.
- [63] Ravi Sandhu, David Ferraiolo, and Richard Kuhn. 2000. The NIST model for role-based access control: Towards a unified standard. In *ACM Workshop on Role-based Access Control*.
- [64] Ravi Sandhu, Kumar Ranganathan, and Xinwen Zhang. 2006. Secure information sharing enabled by trusted computing and PEI models. In *2006 ACM Symposium on Information, Computer and Communications Security*. 2–12.
- [65] Ravi S. Sandhu. 1993. Lattice-based access control models. *Computer* 26, 11 (1993), 9–19.
- [66] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. 1996. Role-based access control models. *Comp.* (1996).
- [67] Daniel Servos and Sylvia L. Osborn. 2017. Current research and open problems in attribute-based access control. *ACM Computing Surveys (CSUR)* 49, 4 (2017), 1–45.
- [68] Jaspreet Singh, Yahuza Bello, Ahmed Refaey Hussein, Aiman Erbad, and Amr Mohamed. 2020. Hierarchical security paradigm for IoT multiaccess edge computing. *IEEE Internet of Things Journal* 8, 7 (2020), 5794–5805.
- [69] Bo Tang, Hongjuan Kang, Jingwen Fan, Qi Li, and Ravi Sandhu. 2019. IoT passport: A blockchain-based trust framework for collaborative Internet-of-Things. In *SACMAT'19*.
- [70] Yuan Tian, Nan Zhang, Yueh-Hsun Lin, XiaoFeng Wang, Blase Ur, Xianzheng Guo, and Patrick Tague. 2017. SmartAuth:User-Centered authorization for the Internet of Things. In *USENIX Security 17*.
- [71] Romans Vanickis, Paul Jacob, Sohelia Dehghanzadeh, and Brian Lee. 2018. Access control policy enforcement for zero-trust-networking. In *2018 29th Irish Signals and Systems Conference (ISSC'18)*. IEEE, 1–6.
- [72] Miao Wu, Ting-Jie Lu, Fei-Yang Ling, Jing Sun, and Hui-Ying Du. 2010. Research on the architecture of Internet of Things. In *2010 3rd International Conference on Advanced Computer Theory and Engineering (ICACTE'10)*, Vol. 5. IEEE, V5–484.
- [73] Zhang Xiaojian, Chen Liandong, Fan Jie, Wang Xiangqun, and Wang Qi. 2021. Power IoT security protection architecture based on zero trust framework. In *2021 IEEE 5th International Conference on Cryptography, Security and Privacy (CSP'21)*. IEEE, 166–170.
- [74] Wenjun Xiong and Robert Lagerström. 2019. Threat modeling – A systematic literature review. *Computers & Security* 84 (2019), 53–69. <https://doi.org/10.1016/j.cose.2019.03.010>
- [75] Guoping Zhang and Jiazheng Tian. 2010. An extended role based access control model for the Internet of Things. In *2010 ICINA*. IEEE.

Received 28 March 2023; revised 22 March 2024; accepted 3 May 2024